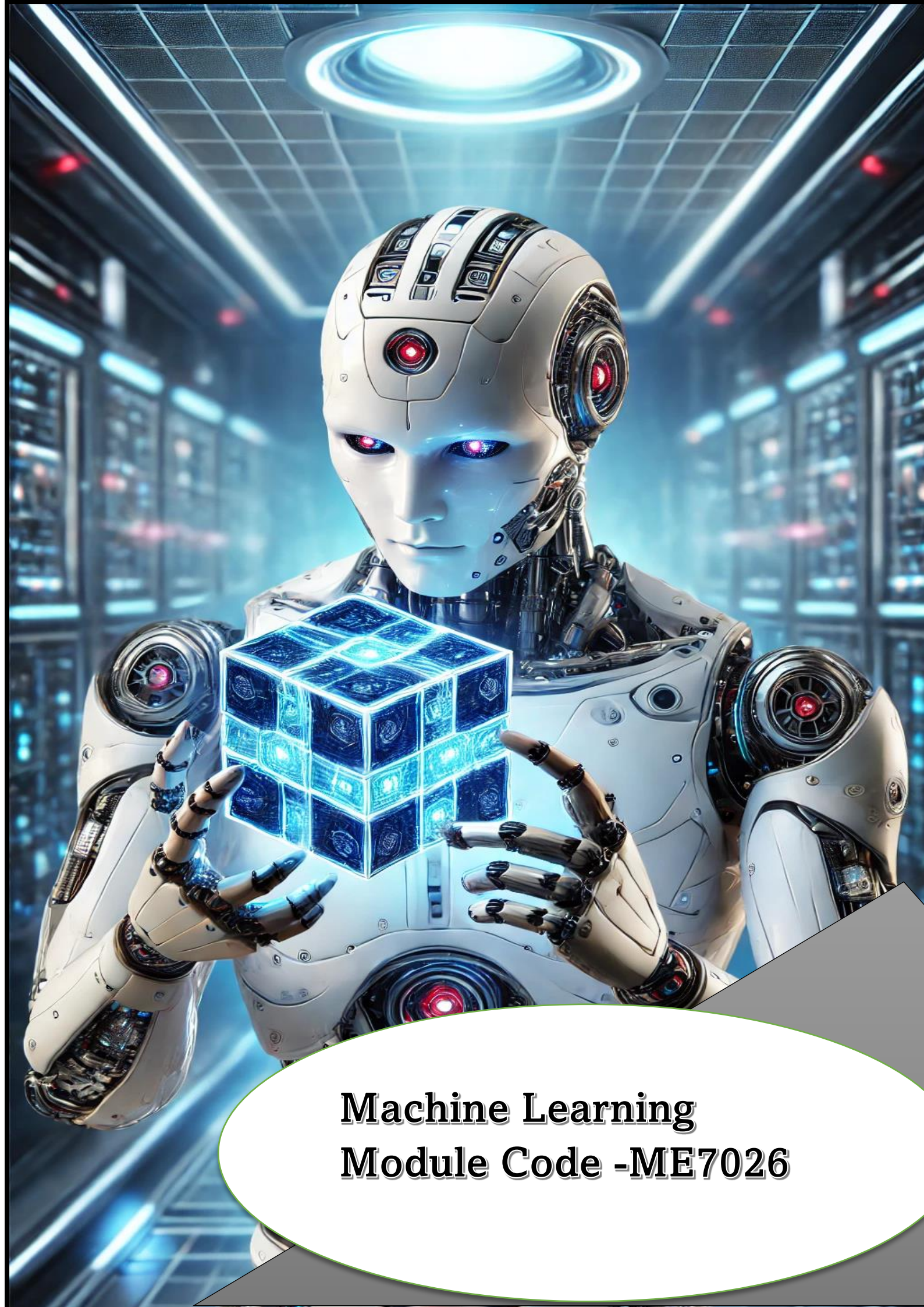




**Machine Learning
Module Code -ME7026**

Machine Learning -Fault classification in Robot Joints

**Kingston
University**
London

**SCHOOL OF ENGINEERING DEPARTMENT OF
MECHANICAL ENGINEERING**

Coursework Assessment Brief

Module Code	ME7026
Module Title	Machine learning
Title of Assessment	Fault classification in Robot Joints
Assessment set by (and contact)	Dr Olga Duran o.duran@kingston.ac.uk

Group Member's Info

K2445563	:	JOSHVA JONATHAN JOSEPH
K2441768	:	OMKAR ANANT JOSHI
K2454461	:	SARATH KUMAR KOMATHUKATTIL
K2447273	:	RAHUL RADHAKRISHNA MENON

Table of Contents

Introduction	4
PART I –Load and overview data	4
PART II – Clustering	6
PART III –Classification: Training and Testing	16
PART IV – Classification using Neural Networks.....	29
References.....	41
Appendix-MATLAB Codes.....	42
Fault classification in Robot Joints- Matlab Codes	42

Machine Learning -Fault classification in Robot Joints

Introduction

This report section underscores the crucial role of understanding the data and its characteristics in the initial steps of loading, exploring, and preparing the Fault Detection in Hexapod Robot Joints dataset for subsequent machine learning tasks, specifically clustering and classification. This understanding ensures that the data is in a suitable format for the chosen algorithms. The primary goal is to create four distinct datasets from the original data, tailored for binary and multiclass classification tasks, and consider smaller sample sizes for efficiency and experimentation.

PART I –Load and overview data

Dataset Loading and Initial Inspection

The dataset, 'Hexapod_One_Joint_data_set-1.csv', was meticulously loaded into the MATLAB environment. It was immediately and thoroughly inspected upon loading to determine its overall structure, data types, and potential issues, such as missing values or inconsistencies. The dataset comprises 17,955 samples, each characterized by 9 features, with an additional column representing the class label.

The features include:

- **X, Y, Z:** Representing spatial coordinates, likely related to the joint's position or movement in three-dimensional space.
- **Slop (x), slop(x/z), slop(1/z), slop(1/x), slop(z/x), slop(z):** These features are derived from the spatial coordinates, potentially representing slopes or rates of change in different directions or combinations.
- **Label:** The target variable indicates each sample's fault type or condition.

Editor - D:\Course work for final\MATFILES\Part1MLm

Command Window

```
X: double
Y: double
Z: double
slop(x): double
slop(x/z): double
slop(1/z): double
slop(1/x): double
slop(z/x): double
slop(z): double
Label: double
```

Statistics for applicable variables:

	NumMissing	Min	Median	Max	Mean	Std
X	0	-640.1993	473.0417	1.0693e+03	427.7195	512.9526
Y	0	0.6583	0.7461	0.7901	0.7362	0.0259
Z	0	-196.7517	-0.2019	180.1845	-0.1086	56.1109
slop_x_	0	-24.8912	-1.6066	1.6843e+03	-0.0582	43.9194
slop_x_z_	0	-1.0333e+04	0.2935	465.7495	37.7547	158.5560
slop_1_z_	0	-184.8087	0.7664	6.1728e+03	24.9712	97.4240
slop_1_x_	0	-575	0.6150	3820	0.8375	30.7440
slop_z_x_	0	-879.5003	0.0022	4.2660e+03	0.0029	36.1920
slop_z_z_	0	-197.2669	0.0034	179.8590	2.1571e-05	3.1533
Label	0	0	1	1	0.9474	0.2233

Number of samples per class:

Class -1: 945 samples

Class 0: 945 samples

Class 1: 945 samples

Class 2: 945 samples

Class 3: 945 samples

Class 4: 945 samples

Class 5: 945 samples

Class 6: 945 samples

Class 7: 945 samples

Class 8: 945 samples

Class 9: 945 samples

Class 10: 945 samples

Class 11: 945 samples

Class 12: 945 samples

Class 13: 945 samples

Class 14: 945 samples

Class 15: 945 samples

Class 16: 945 samples

Class 17: 945 samples

Hexapod dataset summary

- Each class has an equal number of samples (945)

Machine Learning -Fault classification in Robot Joints

This balance is beneficial for training machine learning models as it prevents bias towards any class, enhancing the model's performance. The label variable represents the condition of the joints, 0 represents fault in first joint, 1 represent fault in 2nd joint and so on, increasing sequentially up to 17, whereas -1 represents all joints are fault free.

Data Preparation: Creating the Four Datasets

The given Hexapod Joint dataset is used to generate the following four datasets:

a) Binary Output (Fault/No Fault) Dataset:

This dataset was created to simplify the classification problem into a binary task: fault versus no fault. The original 'Label' column, with distinct classes, was transformed into a binary label. This was achieved by categorising one class as 'Non-Faulty Cases'(0) and grouping the remaining classes into a single 'Faulty Cases'(1) category. These resulted in a dataset with two classes:

- Non-Faulty Cases: 945 samples
- Faulty Cases: 17,010 samples

The resulting dataset, named 'binaryData', consists of 17,955 rows and 10 columns (9 features + 1 binary label). This represents highly imbalanced dataset, with significantly more faulty than non-faulty cases. This factor must be considered when clustering and training classification models in subsequent parts of the assignment. Strategies such as oversampling the minority class or under sampling the majority classes can be used.

b) Multiple Classifications for Different Types of Faults:

This dataset preserves the original 19 fault classifications as they were in the original dataset. This dataset is intended for multiclass classification tasks, where the goal is to distinguish between the different types of faults affecting the hexapod robot joint. The dataset is saved to a CSV file named 'multiclass_classification.csv' for further use.

c) Binary Output (1% Random Samples):

A random sample of 1% of the original data was selected to create this dataset. The same binary label transformation as in (a) was applied to this subset, resulting in a smaller dataset with binary fault/no fault labels. The primary purpose of this dataset is to enable faster experimentation and prototyping of machine learning models, particularly during the initial stages of development. The 1% dataset has a similar class distribution to the entire dataset but with significantly fewer samples, making it easier to iterate on model design and parameter tuning. The generated dataset was saved as 'binary_classification_2.csv'.

d) Multiple Classifications (1% Random Samples):

Like (c), this dataset was created by randomly sampling 1% of the original data. However, in this case, the original 19 fault classifications were retained. This dataset provides a smaller, more manageable dataset for experimenting with multiclass classification algorithms.

Statistical Summary

- The number of Missing Values indicates the presence of any missing data points, which could require imputation or removal.

Machine Learning -Fault classification in Robot Joints

- **Minimum and Maximum Values:** Provide the range of values for each feature, which can help identify outliers or scaling requirements.
- **Median:** The middle value in the distribution is less sensitive to outliers than the mean.
- **Mean:** The average value, providing a measure of central tendency.
- **Standard Deviation (or σ):** it's a measure of the spread or variability of the data around the mean.

The statistical analysis produced the following results:

- **Absence of Incomplete Data:** The data is complete, showing no missing attribute values. This completeness makes the data preprocessing step easier, making imputation techniques unnecessary.
- **Diversity of Variables:** Unique features have different ranges of values. For example, the 'X' feature ranges from -640.1993 to 1069.3, while the 'Y' feature is limited to a very narrow band of 0.6583 to 0.7901. This difference in the ranges suggests that feature scaling may be critical to counter the influence of features with wider ranges on the learning algorithm.
- **Feature Distributions:** Analysing the mean and median values, as well as the standard deviation, provides insights into each feature's distribution. For example, a significant standard deviation indicates a broader range of values, while a significant difference between the mean and median suggests an uneven distribution,

PART II – Clustering

K-Means

Binary dataset

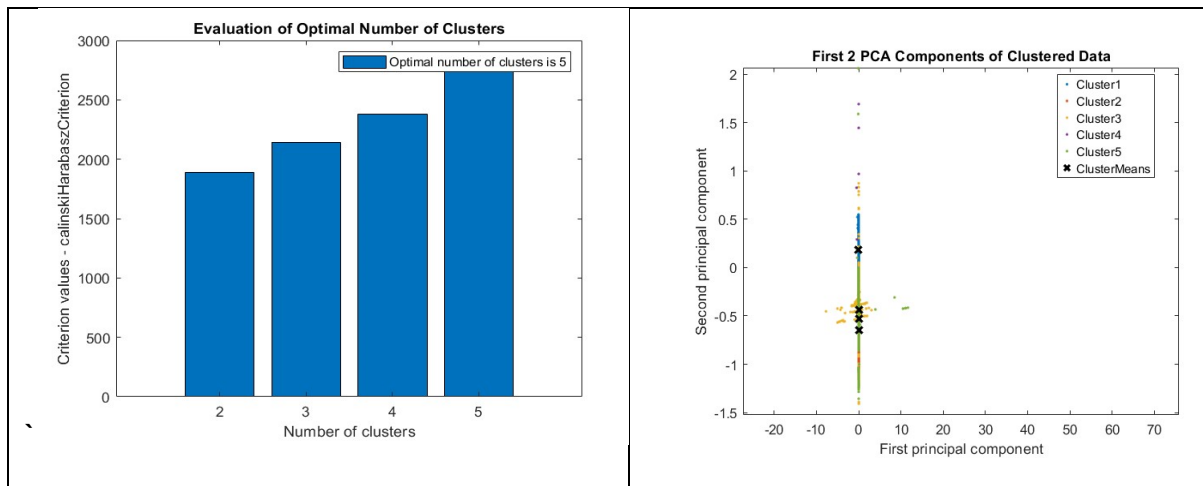
Since the dataset provided to us was quite large, and K-means is suitable for distinguishing faulty patterns, we decided to use K-means as our first clustering method. The centroids offer an easy-to-understand representation of the clusters. In this section, our first goal was to apply K-means clustering to all the types of datasets. We started by using K-means clustering for binary dataset.

To determine the optimal number of clusters, we use the **Calinski-Harabasz** criterion to select the best K value, and the optimum number of clusters was found to be 5.

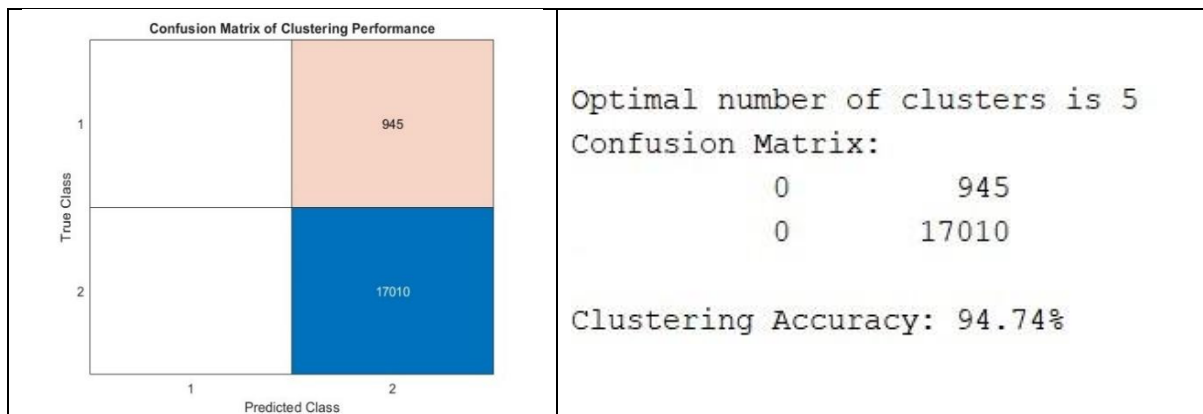
Applied K-Means clustering using “**Correlation**” distance as a clustering method and computed cluster centroids to understand the structure of clusters.

We used PCA (Principal Component Analysis) and generated a scatter plot to display the components of clustered data.

Machine Learning -Fault classification in Robot Joints



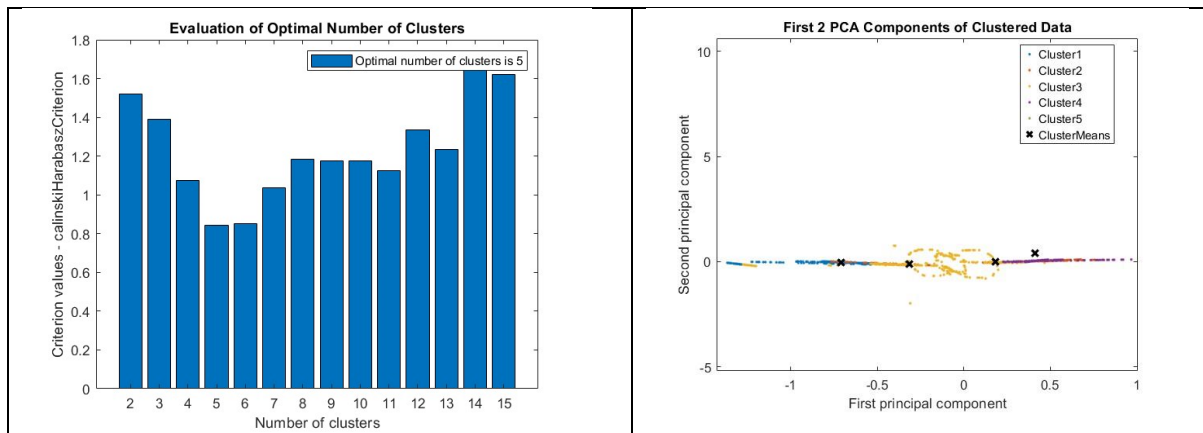
As we can observe, the clustering result does not appear to be very good, there is a significant overlapping of clusters around the centre. Most data points are clustered around the origin, indicating PCA might not be effectively differentiating the clusters.



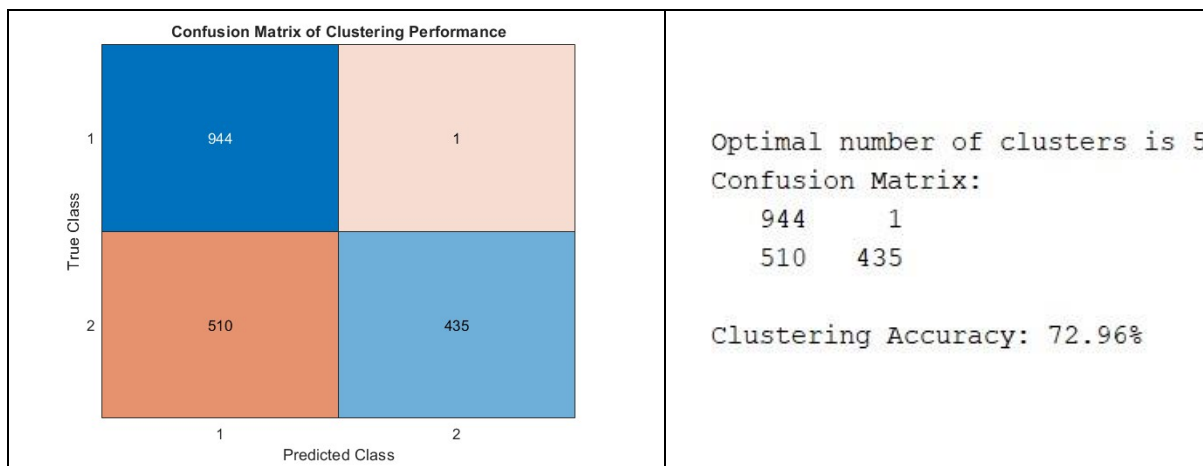
Even though the confusion matrix states that the clustering accuracy is 94.74%, we know that clustering was not good enough. The clustering algorithm performed well in detecting fault cases but completely failed to classify non-faults. The data set imbalances (fault cases significantly outnumbering non-fault cases) might have affected clustering performances, making it harder for the model to form a distinct cluster for the minority classes.

We wanted to overcome the class imbalance issue and improve clustering accuracy by handling class imbalance. The dataset was imbalanced, with faulty cases 1 significantly outnumbering non-faulty cases 0. We tried under sampling the majority class to match the number of non-fault cases by randomly selecting a subset of fault cases to create a balanced dataset. We normalized the balanced dataset as well before applying clustering to get a better result. This time, we tried the **Davies-Bouldin Index** to evaluate different K values.

Machine Learning -Fault classification in Robot Joints



The images indicate some improvement in clustering however, the enhancement is not significant, as we can still see overlapping and most data points aligned in a narrow horizontal band.



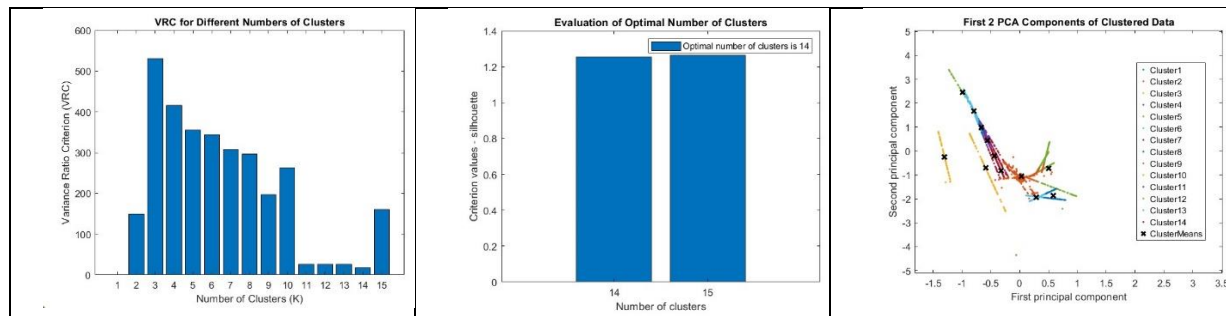
This confusion matrix demonstrates a reasonable separation between faults and non-faults. However, the accuracy is only 74%, indicating that the classification is not entirely precise. Most of the no-fault cases have been correctly classified, while more than 40% of the fault cases have also been correctly identified

As under-sampling alone didn't give the best results, we tried using the SMOTE function and under-sampling together to balance the dataset.

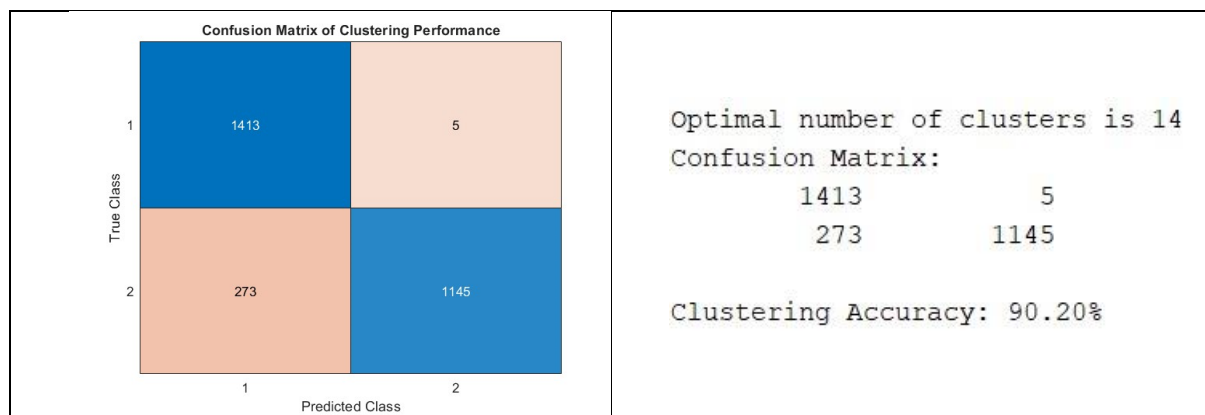
As we got a better result using the Davies-Bouldin index, we stuck to it as it provides a solid evaluation strategy for choosing the optimum number of clusters, which made clustering more robust and less likely to suffer from over- or under fitting.

Z-score normalisation improved the clustering process, especially when using K means, and we also evaluated the confusion matrix with the ground truth to assess clustering results. This structured approach provided us with the best possible solution for this binary data set by efficiently separating the two different classes.

Machine Learning -Fault classification in Robot Joints



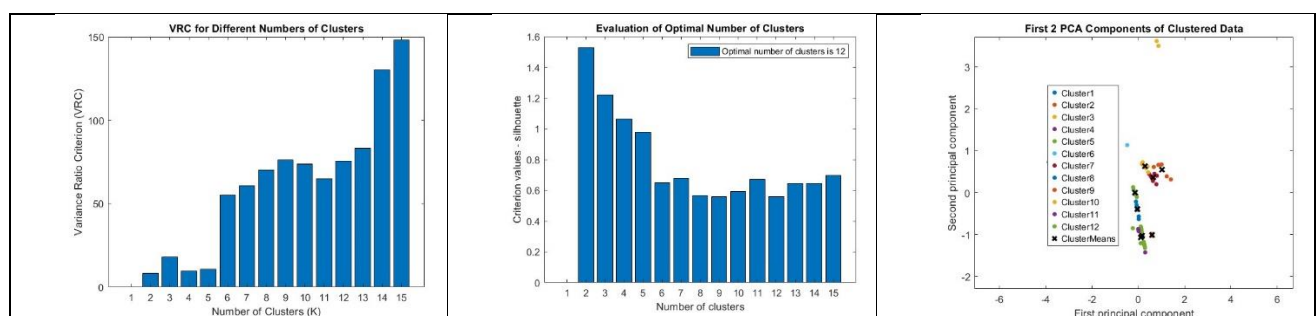
There has been some significant improvement in the clustering results, the clustering appeared to be well structured with minimal overlap and well-positioned centroids. Most clusters appear to be relatively close, suggesting that points within the same cluster are closely packed.



The latest clustering results show a 90% accuracy in distinguishing between faults and non-faults, with a well-separated cluster structure, which is a significant improvement. The selection of 14 clusters has contributed to a more balanced distribution of cases, reducing classification error and enhancing overall clustering performance.

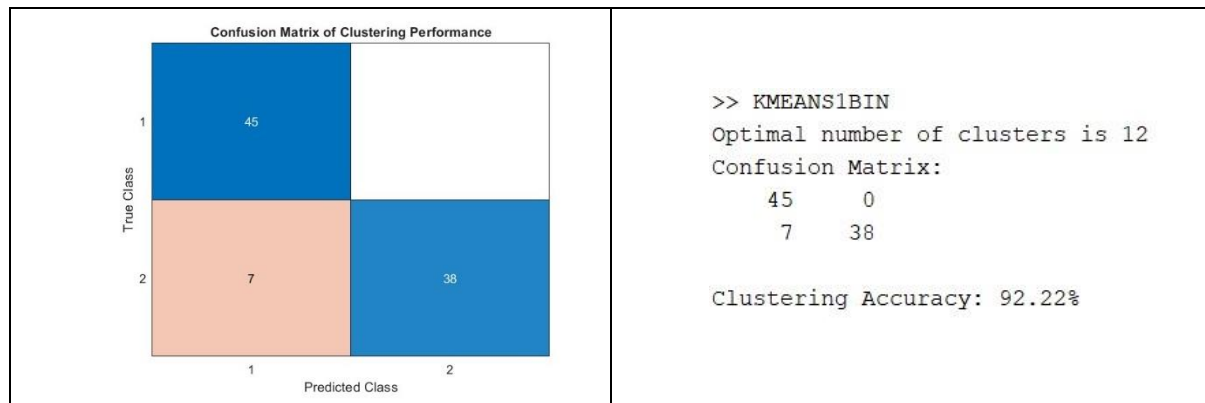
1% Binary dataset

With the same parameter configurations that we used for the Binary dataset, we used a 1% Binary dataset as well, and we achieved good clustering results.



The above three images gives us an overview of optimal clusters and plot graphs of K means clustering.

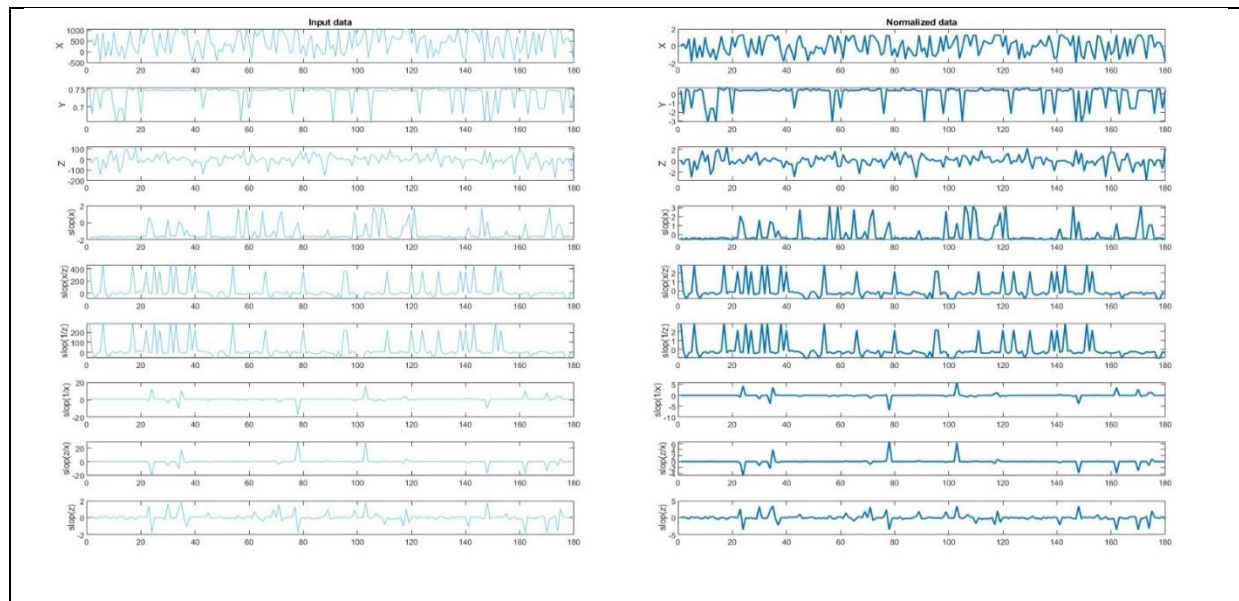
Machine Learning -Fault classification in Robot Joints



The above confusion matrix shows a good separation between classes with a clustering accuracy of 92.22%.

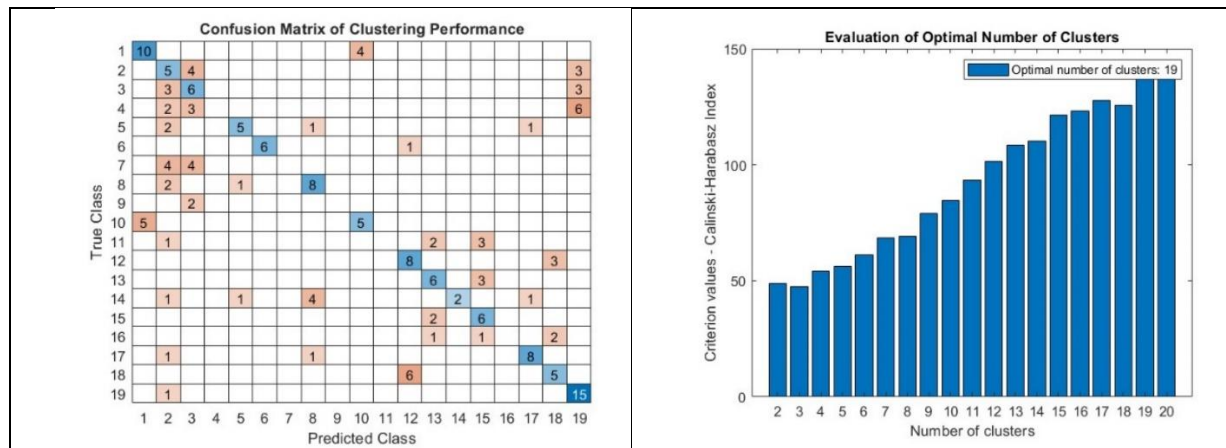
1% Multiclass dataset

We tried K-means clustering with a multiclass dataset, and we didn't get good clustering results right away, so we tried normalizing each feature with Z-score, excluding the labels, which prepared the dataset for clustering by scaling it to a standard range, and we visualised both the original and normalized data.



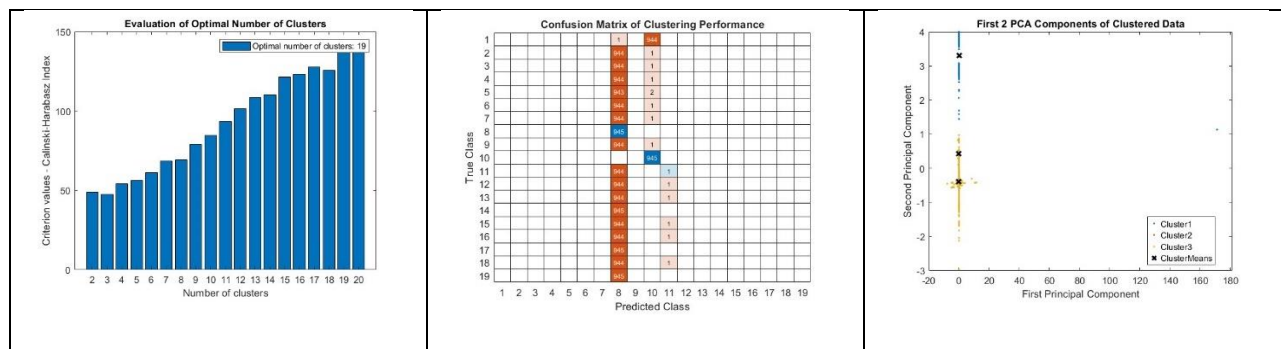
We have tried techniques like PCA to remove noise, which can distort clustering, and we also tried using SMOTE to balance the data. We tried refining clustering algorithm parameters, but we were not able to achieve a good clustering result and tried cluster post-processing as it refines the label mapping process to ensure that the predicted clusters are matched accurately with true labels. We considered using weighted voting to improve label assignments and reduce errors, still there were no improvements in clustering results. We were only able to reach 53.33% clustering accuracy.

Machine Learning -Fault classification in Robot Joints



Multiclass dataset

We got even worse results with full multiclass datasets with K-means



DBSCAN

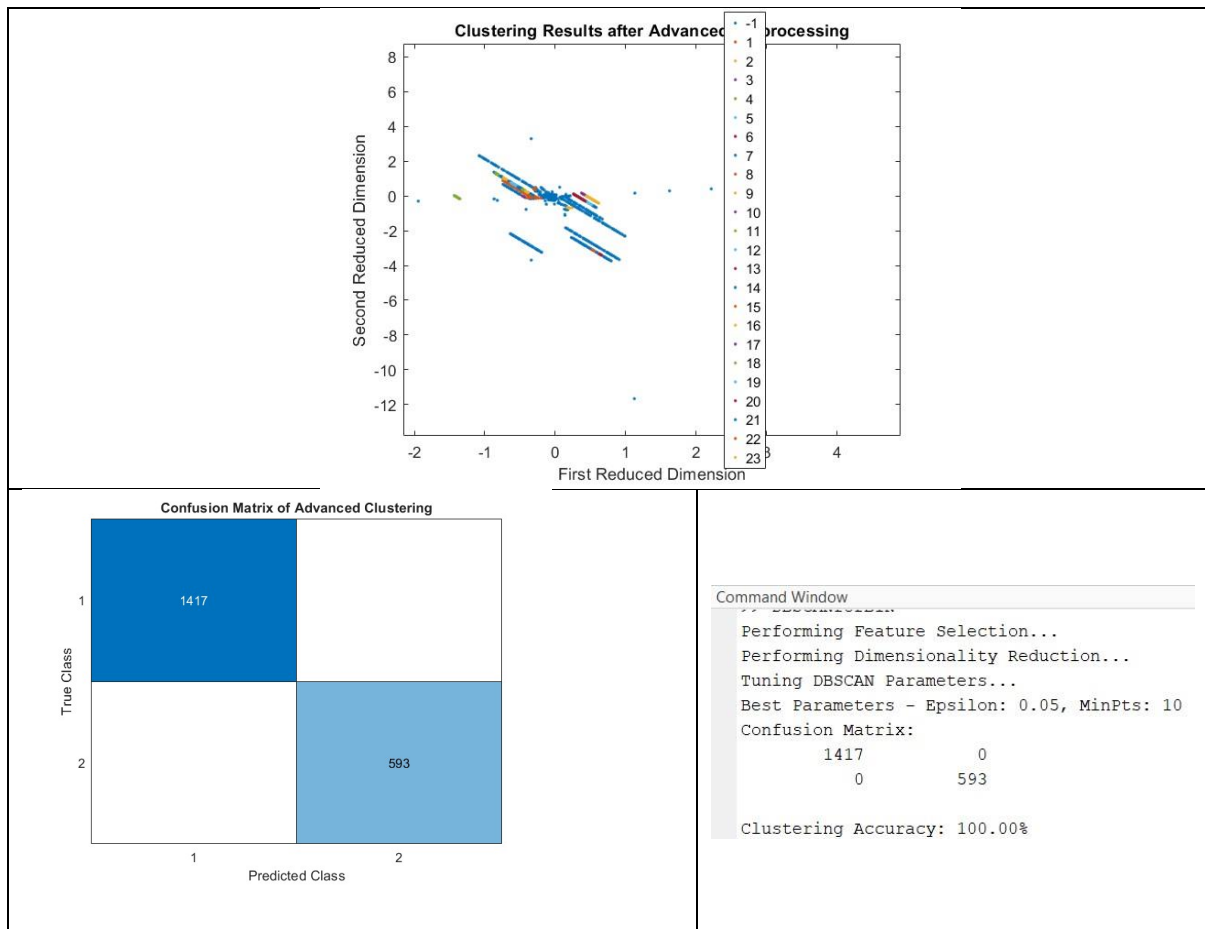
We opted for **DBSCAN** as our second clustering algorithm because it is great for detecting faults from irregular patterns, and it also does not require you to specify the number of clusters like **K-means**. It is great for detecting errors in hexapod robot joints.

Binary dataset

For binary dataset, we started by initial data preparation that is extracting features and labels from the input data separating last columns as labels and remaining columns as features and we use SMOTE to address class imbalance by oversampling the minority classes and under sampling majority classes to ensure equal representation of classes.

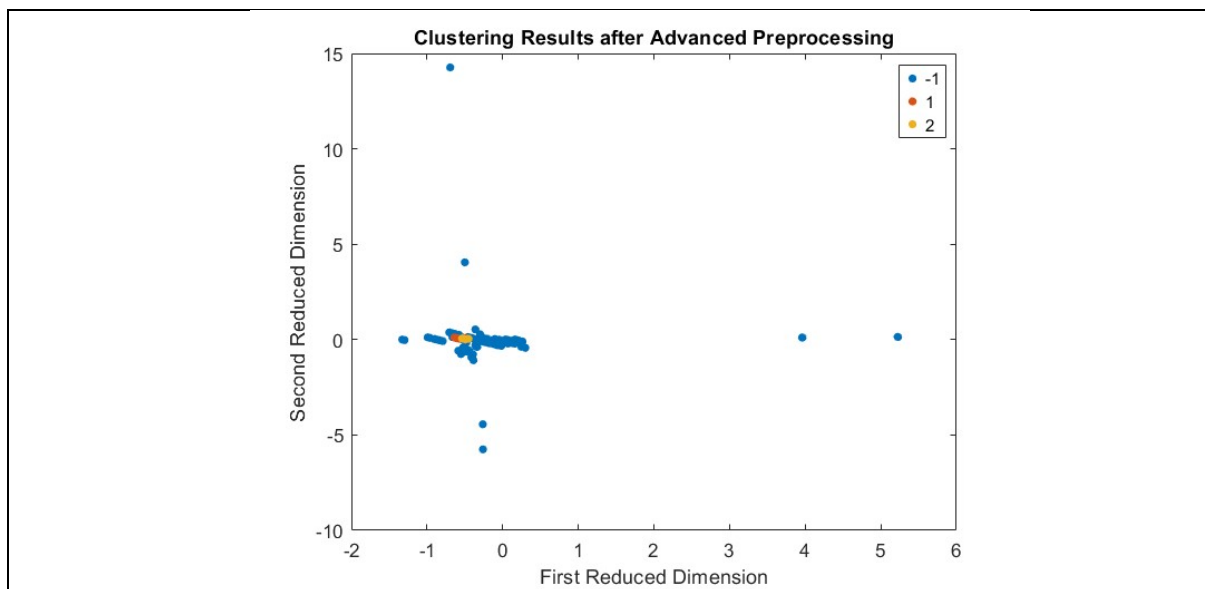
Applied Z-score standardisation to features and transformed the data to have zero mean and unit variance. We used the relief algorithm for feature ranking and selecting features based on importance. For parameter tuning, we explored values 0.05 to 1 and tested minimum points 10,15,20,25,30 and used cross-validation to find best parameters.

Machine Learning -Fault classification in Robot Joints

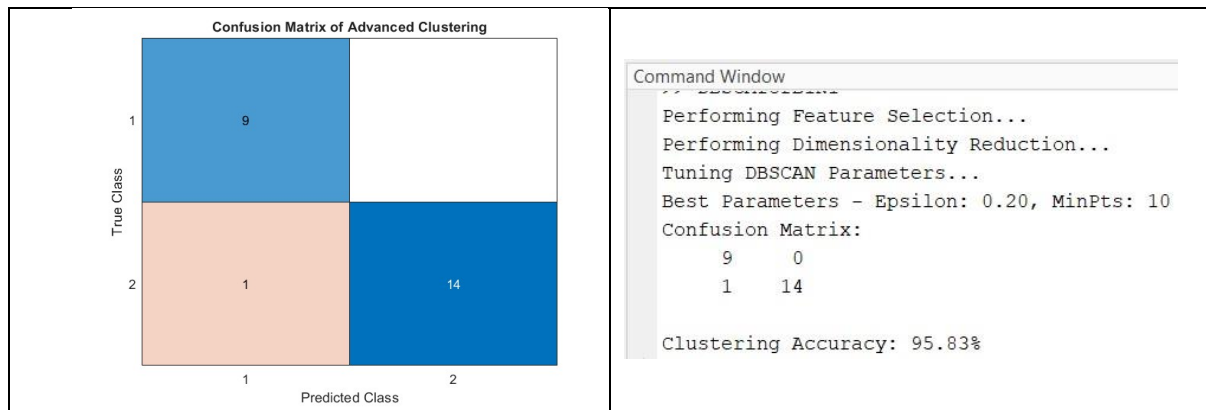


We got a great clustering accuracy of 100% with perfect separation between faults and no-faults.

1% Binary dataset



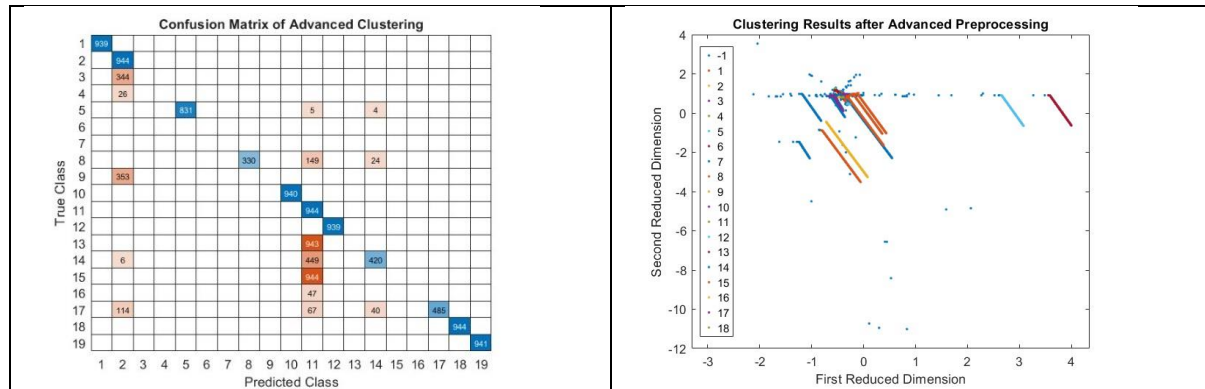
Machine Learning -Fault classification in Robot Joints



Multiclass Dataset

The multiclass dataset has 19 classes with 945 samples each, there are 18 different types of faults that goes from 0 to 17 and non-fault represented by -1. Since it has multiple classes, it is difficult to classify each and every class perfectly but, the DBSCAN clustering algorithm is a suitable clustering algorithm as it handles multiple classes, and it is most commonly used algorithm to detect faults in robot joints. We used DBSCAN algorithm as follows

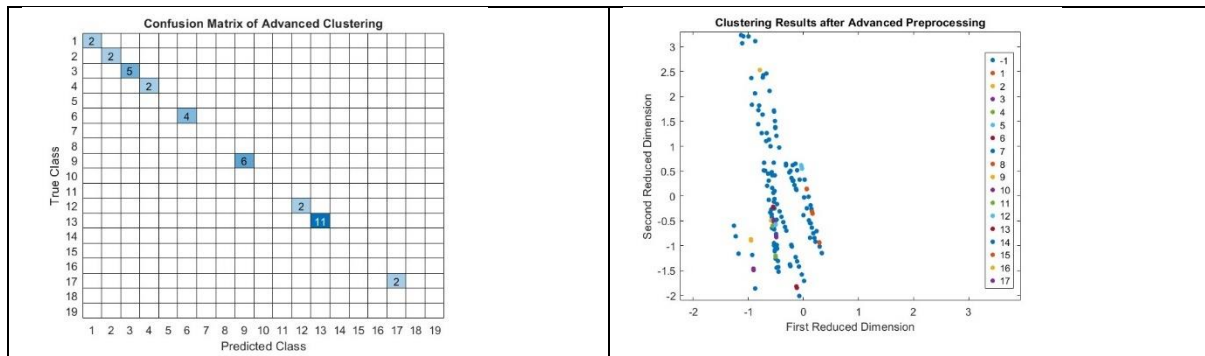
We have loaded the features and labels from multiclass data, standardise features using Z-score to normalise the data. Performed PCA and reduced data to top 5 principle components for best clustering efficiency.



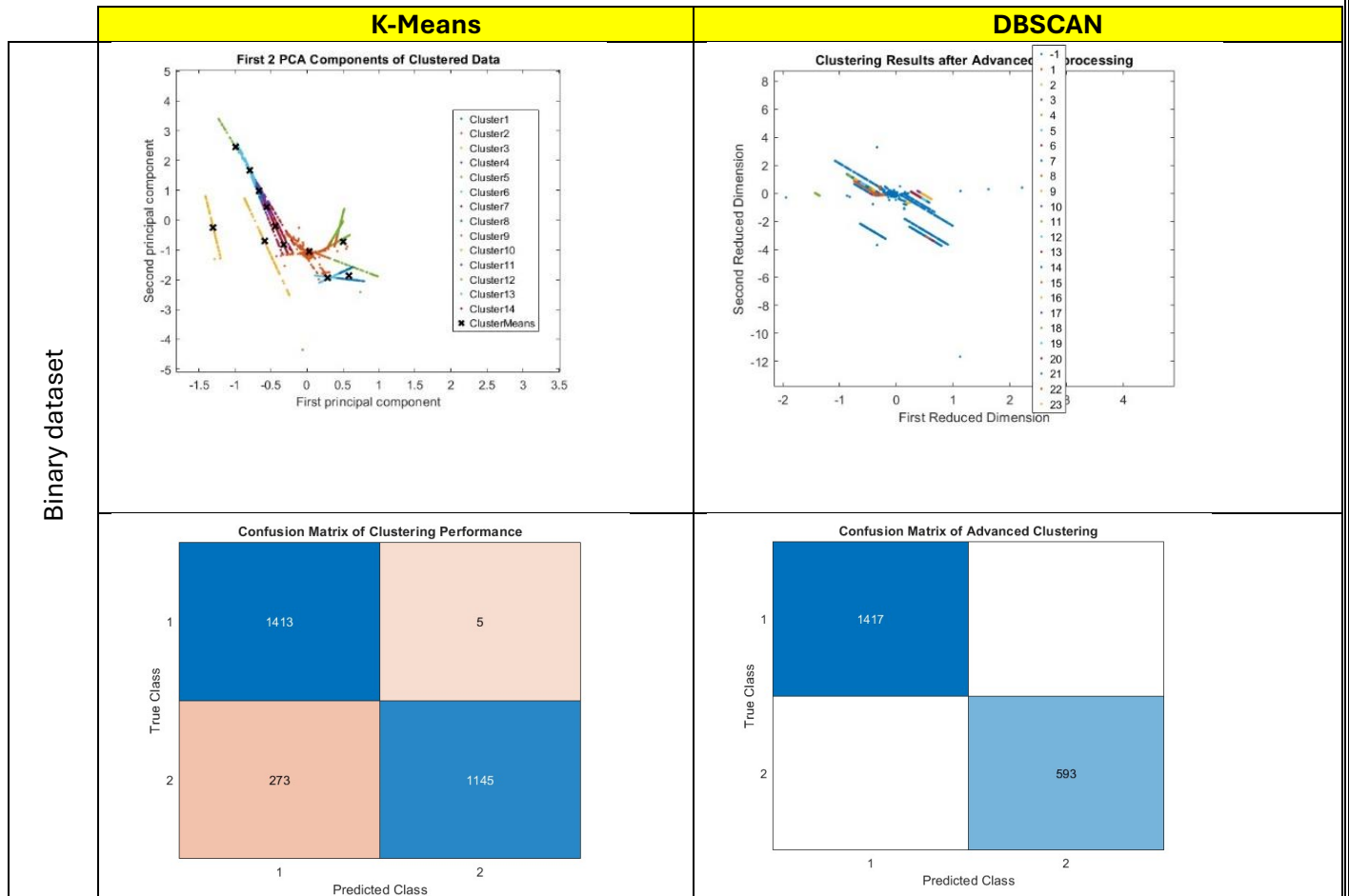
The best accuracy we possibly got was 74% even though there were some good classifications there were also some miss classifications and we found it difficult to make any improvements to the dataset.

Machine Learning -Fault classification in Robot Joints

1% Multiclass Dataset



Comparison between two clustering methods.



Machine Learning -Fault classification in Robot Joints

Optimal number of clusters is 14

Confusion Matrix:

1413	5
273	1145

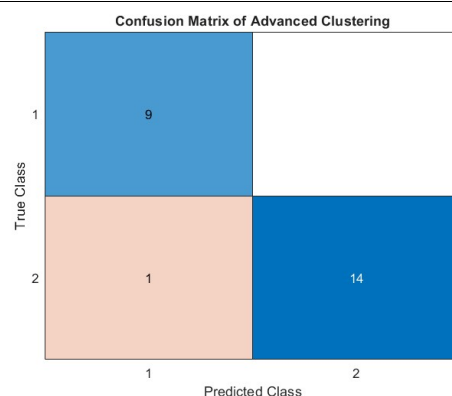
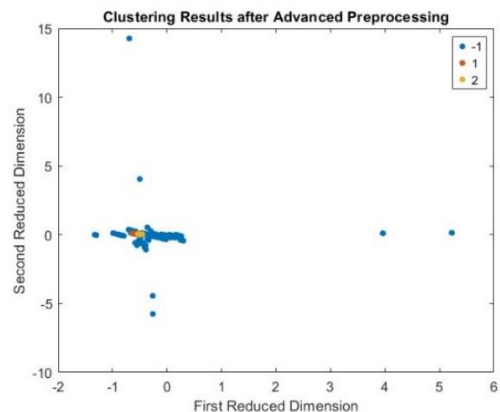
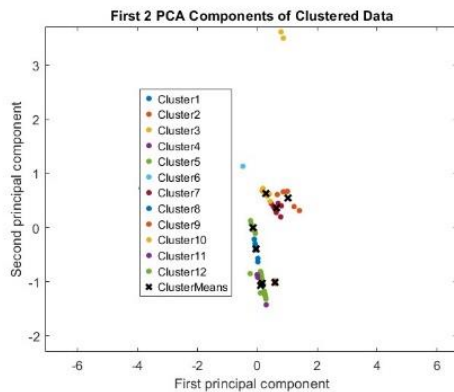
Clustering Accuracy: 90.20%

Command Window

```
>> DBSCAN
Performing Feature Selection...
Performing Dimensionality Reduction...
Tuning DBSCAN Parameters...
Best Parameters - Epsilon: 0.05, MinPts: 10
Confusion Matrix:
    1417      0
      0    593
Clustering Accuracy: 100.00%
```

For the Binary Dataset DBSCAN provided even better result then K-means Clustering algorithm

1% Binary dataset



>> KMEANS1BIN

Optimal number of clusters is 12

Confusion Matrix:

45	0
7	38

Clustering Accuracy: 92.22%

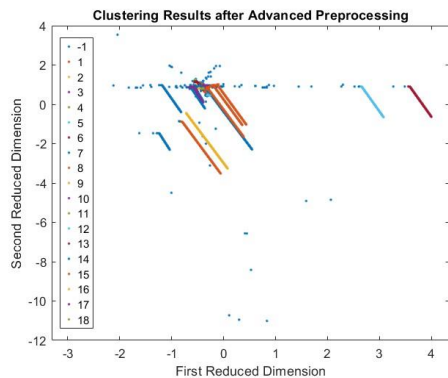
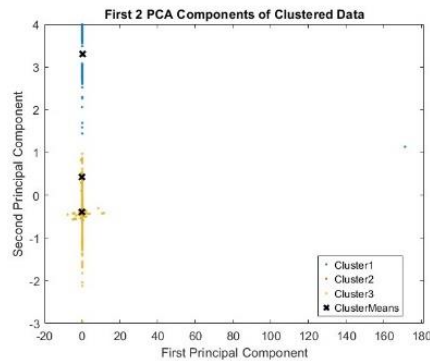
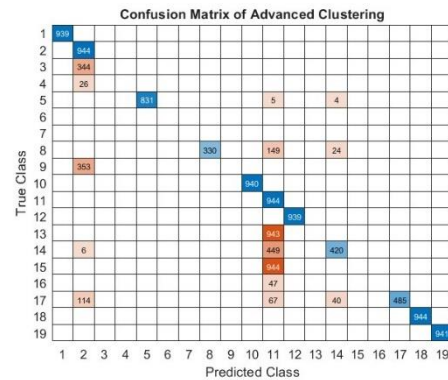
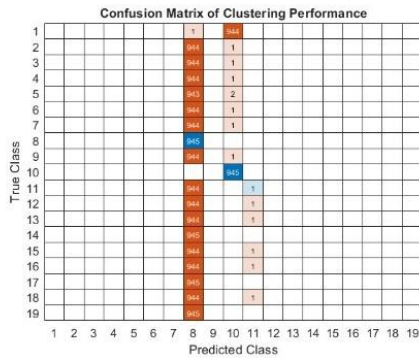
Command Window

```
>> DBSCAN
Performing Feature Selection...
Performing Dimensionality Reduction...
Tuning DBSCAN Parameters...
Best Parameters - Epsilon: 0.20, MinPts: 10
Confusion Matrix:
      9      0
      1     14
Clustering Accuracy: 95.83%
```

For the 1% Binary Dataset DBSCAN provided even better result then K-means Clustering algorithm

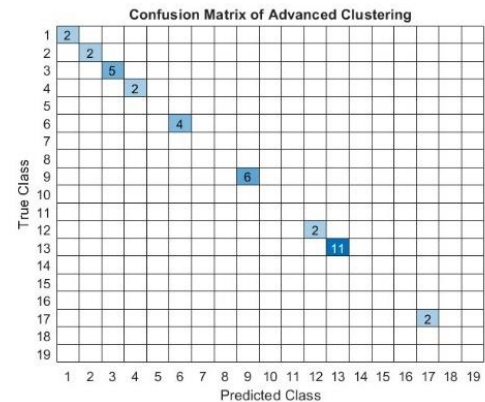
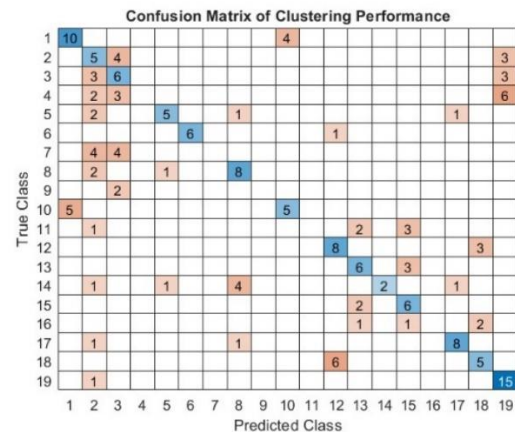
Machine Learning -Fault classification in Robot Joints

Multiset Dataset



For the Multiclass Dataset DBSCAN provided even better result then K-means Clustering algorithm

1% Multiset Dataset



For the 1%Multiclass Dataset K-means provided even better result then DBSCAN Clustering algorithm

Conclusion:

After comparing the confusion matrix of each data set DBSCAN clustering algorithm was found to be the most suitable one for clustering and handling large datasets, it was more reliable and accurate then K-means Clustering algorithm.

PART III –Classification: Training and Testing

Classification for the binary and multiclass dataset is done by using classification learner tool: -

A classification learner is a piece of software or system that automates training, testing, and finding the best classification model for a given dataset. It provides an easy-to-use interface

Machine Learning -Fault classification in Robot Joints

for running, testing, and comparing machine-learning algorithms based on performance metrics.

1)Types of methods attempted

We have explored various classification methods to achieve the best results for this dataset. For both binary and multiclass data, we employed four types of classification techniques. Tree-based models, such as Fine Trees and Boosted Trees, often outperform Linear Support Vector Machines (SVM) and Linear Discriminant Analysis (LDA) (Feldesman, 2002).Tree-based models are effective at capturing complex nonlinear patterns and excel at handling noise and outliers due to their threshold-based splitting mechanism. A key advantage of tree-based classification is its ability to manage imbalanced data by focusing on misclassified samples. Since our dataset is highly imbalanced in a binary format, tree-based classification is the most suitable method

2) Evaluation (Split Data)

Train-Test Split: - 80-20

Metrics used – Accuracy, F1 Score, ROC AUC Score & Confusion Matrix

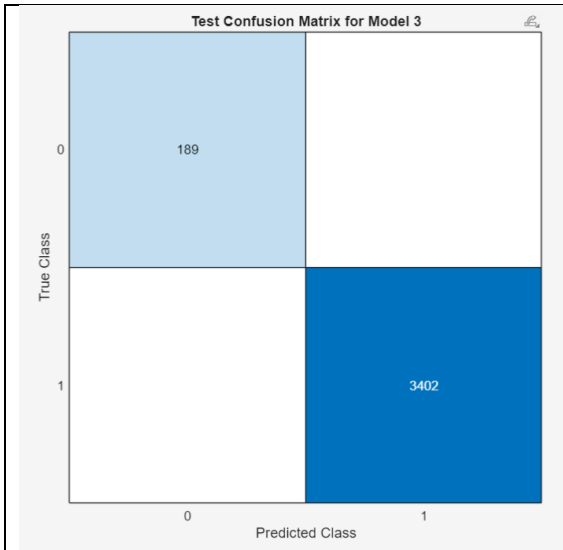
1.) Binary Data

Binary classification in fault detection aims to differentiate between the fault and no fault in hexapod robot joints and below table gives the results: -

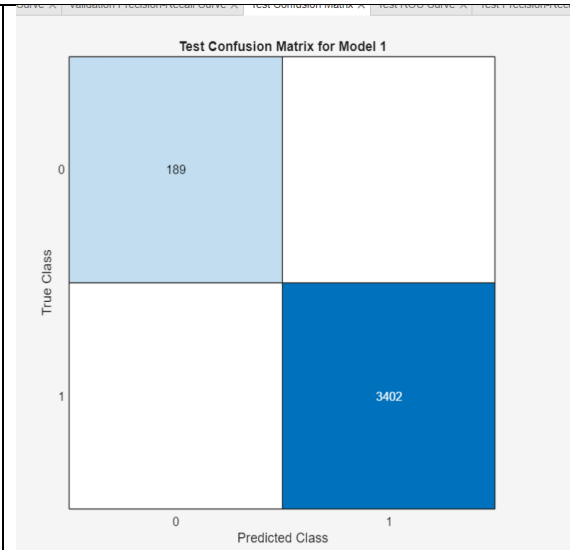
Classification Types	Accuracy	F1-Score	AUC-ROC
Fine tree	99.99	0.8553	0.76
Linear SVM	94.74	0.9730	0.91
Fine KNN	99.97	1.0000	1.0
Boosted Trees	94.74	0.8650	0.85

a) Below we will be mentioning the confusion matrix and f1 scores for the two best classification methods used for binary dataset

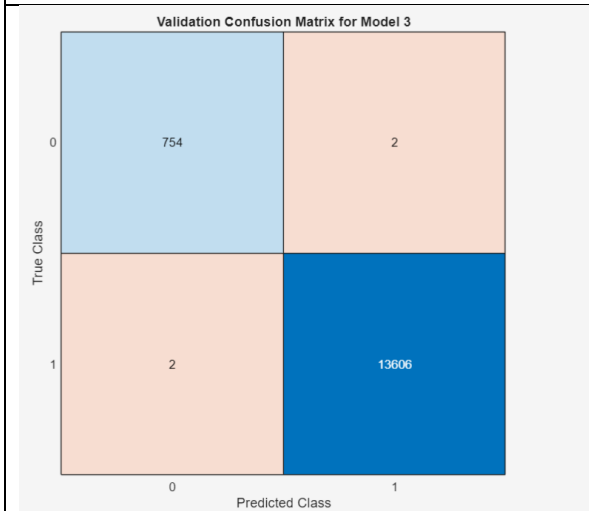
Machine Learning -Fault classification in Robot Joints



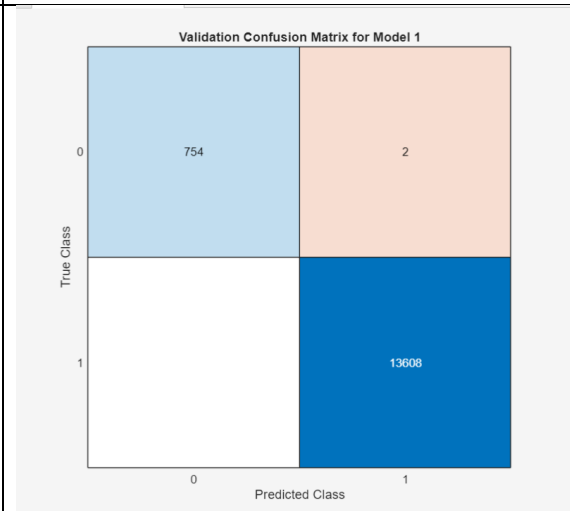
Test Confusion Matrix (Fine KNN)



Test Confusion Matrix (Fine Trees)



Validation Confusion Matrix (Fine KNN)



Validation Confusion Matrix (Fine Trees)

```

1  TP = 3402;
2  FP = 189;
3  precision = TP / (TP + FP);
4  FN = 0;
5  recall = TP / (TP + FN);
6  F1_score = 2 * (precision * recall) / (precision + recall);
7  fprintf('F1-score: %.4f\n', F1_score);
8  if isnan(F1_score)
9      F1_score = 0;
10 end
11
Command Window
>> F1ScoreBinaryClassLinearSVM
F1-score: 0.9730
    
```

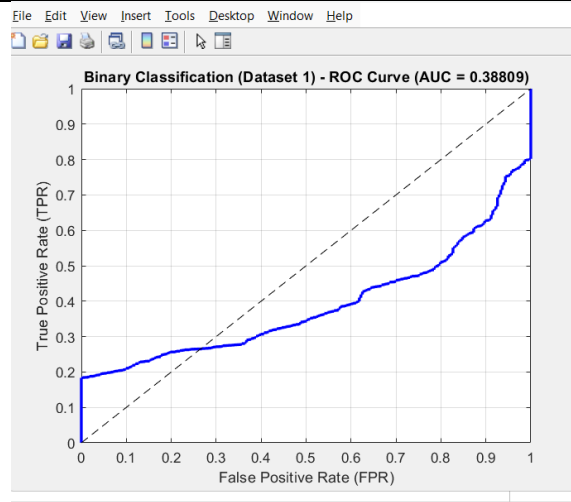
F1 Score (Linear SVM)

```

1  TP = 3402;
2  FP = 0;
3  precision = TP / (TP + FP);
4  FN = 0;
5  recall = TP / (TP + FN);
6  F1_score = 2 * (precision * recall) / (precision + recall);
7  fprintf('F1-score: %.4f\n', F1_score);
8  if isnan(F1_score)
9      F1_score = 0;
10 end
11
Command Window
>> F1ScoreBinaryClassKnn
F1-score: 1.0000
    
```

F1 Score (Fine KNN)

Machine Learning -Fault classification in Robot Joints



ROC AUC Curve (Binary Data)

Conclusion

From the above results, we can predict that Fine Tree and Fine KNN gives the best classification in comparison with the other two classification methods of the binary data set.

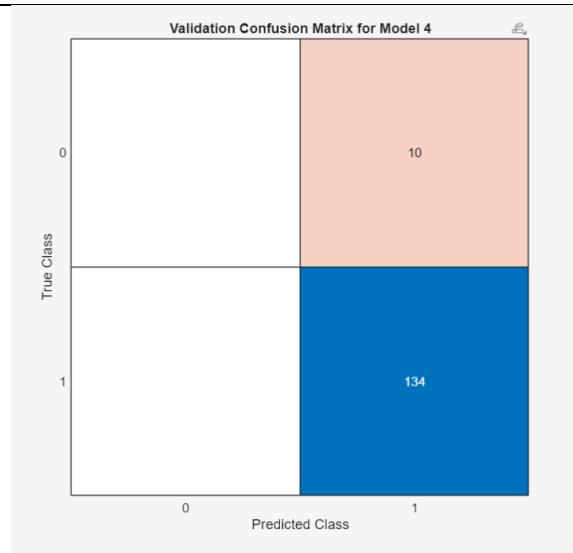
2.) Binary Subset Data

Binary classification in fault detection aims to differentiate between the fault and no fault in hexapod robot joints and below table gives the results: -

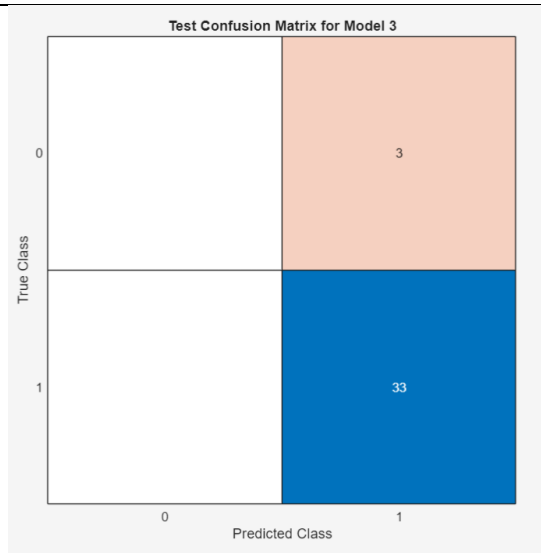
Classification Types	Accuracy	F1-Score	AUC-ROC
Fine tree	100.00	1.0000	1.0
Linear Discriminant	91.67	0.6532	0.67
Fine KNN	93.07	0.7359	0.74
Boosted Trees	93.07	0.9640	0.92

b) Similarly, we have mentioned the confusion matrix and f1 scores for the two best classification methods used for binary subset dataset

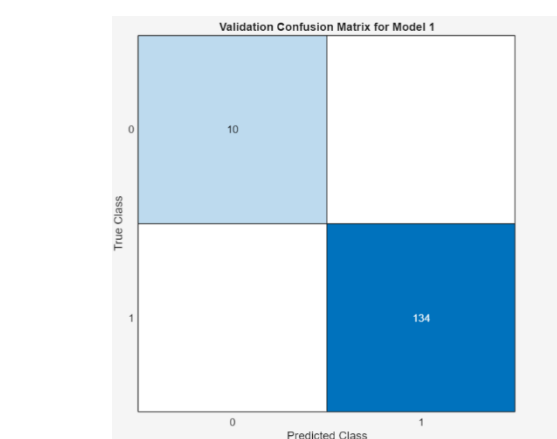
Machine Learning -Fault classification in Robot Joints



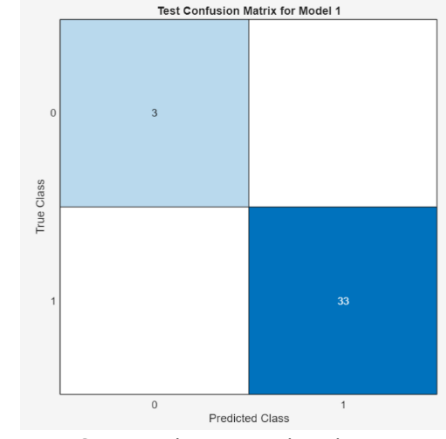
Validation Confusion Matrix (Boosted Trees)



Test Confusion Matrix(Boosted Trees)



Validation Confusion Matrix (Fine Trees)



Test Confusion Matrix(Fine Trees)

```

1 % Define the confusion matrix
2 TP = 134; % True Positives
3 FP = 0; % False Positives
4 FN = 0; % False Negatives
5 TN = 10; % True Negatives
6
7 % Calculate Precision and Recall
8 Precision = TP / (TP + FP);
9 Recall = TP / (TP + FN);
10
11 % Calculate F1-score
12 F1_score = 2 * (Precision * Recall) / (Precision + Recall);
13
14 % Display results
15 fprintf('Precision: %.2f\n', Precision);
16 fprintf('Recall: %.2f\n', Recall);
17 fprintf('F1-score: %.2f\n', F1_score);
18

```

Command Window

```

>> Precision: 1.00
>> Recall: 1.00
>> F1-score: 1.00

```

F1 Score(Fine Trees)

```

1 TP = 134;
2 FP = 0;
3 precision = TP / (TP + FP);
4 FN = 0;
5 recall = TP / (TP + FN);
6 F1_score = 2 * (precision * recall) / (precision + recall);
7 fprintf('F1-score: %.4f\n', F1_score);
8 if isnan(F1_score)
9     F1_score = 0;
10 end
11

```

Command Window

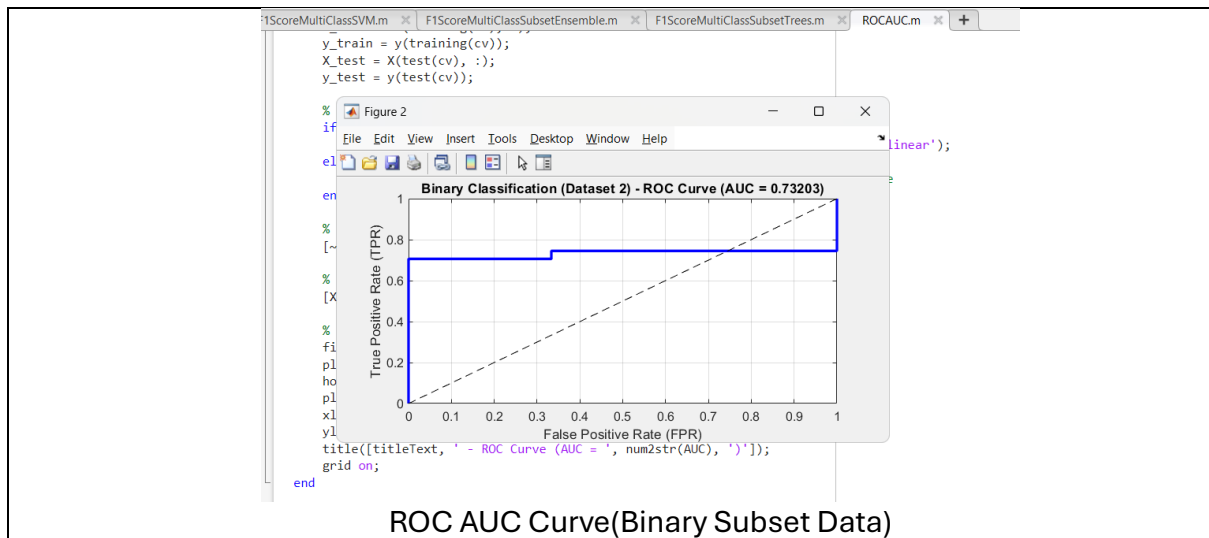
```

>> F1ScoreBinaryClassSubstEnsemble
>> F1-score: 0.9640

```

F1 Score(Boosted Trees)

Machine Learning -Fault classification in Robot Joints



Conclusion

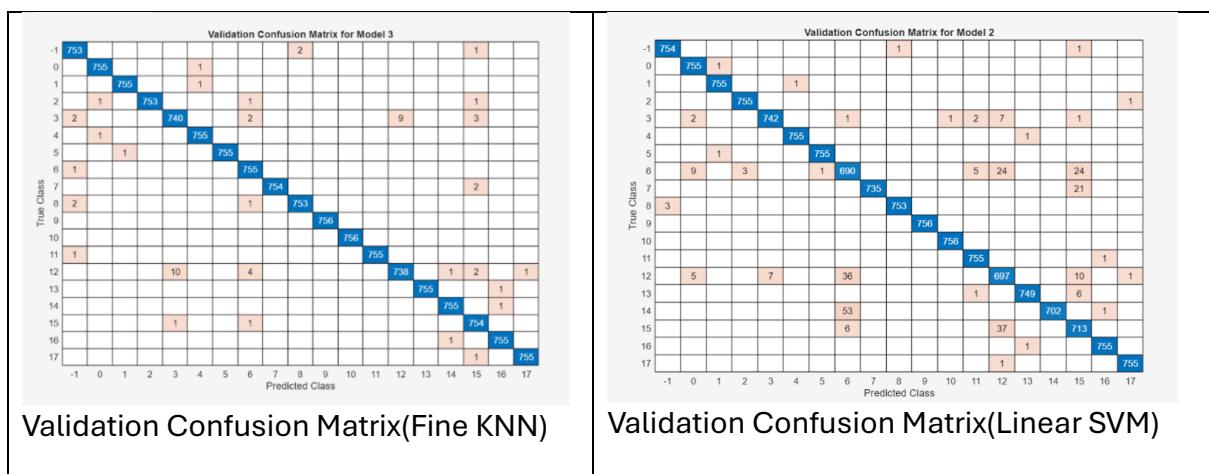
From the above results we can conclude that Fine Trees and Boosted Trees(Ensemble) give the best classification in comparison with the other two for binary subset data

3.) Multiclass Data

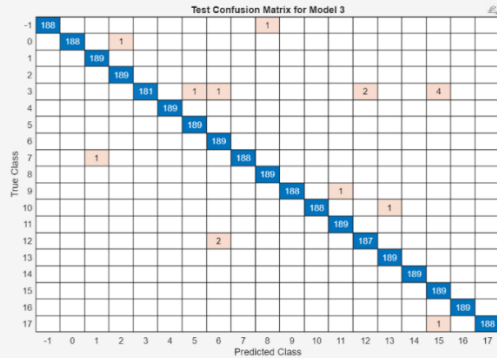
Multiclass classification in fault detection aims to differentiate between all the different types of faults in hexapod robot joints and below table gives the results: -

Classification Types	Accuracy	F1-Score	AUC-ROC
Fine tree	98.76	0.9956	1.000
Linear SVM	98.07	0.998	0.9765
Fine KNN	99.60	0.9987	0.9259
Boosted Trees	98.14	0.9987	0.9526

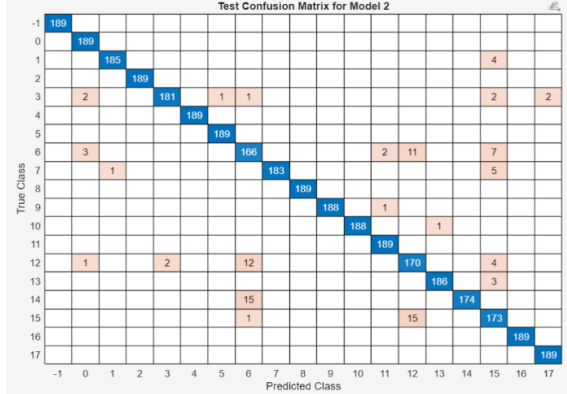
c) Similarly, we have mentioned the confusion matrix and f1 scores for the two best classification methods used for multiclass dataset



Machine Learning -Fault classification in Robot Joints



Test Confusion Matrix(Fine KNN)



Test Confusion Matrix (Linear SVM)

```

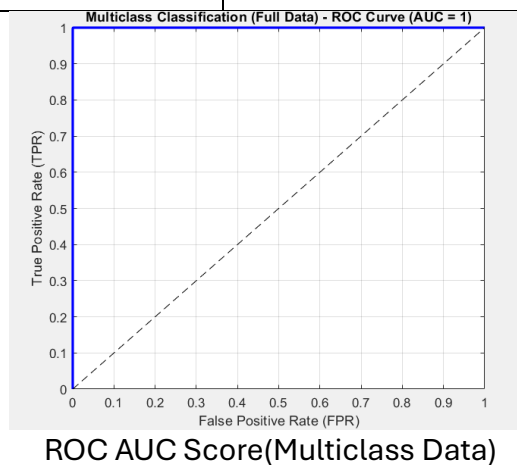
49 disp(recall);
50 disp('class-wise F1-score:');
51 disp(f1_scores);
52 disp(['Weighted F1-score: ', num2str(weighted_f1)]);
53
54 end
55 confMatrix = [753 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
56 2 755 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
57 1 1 753 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
58 2 1 2 740 2 9 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
59 1 0 0 2 755 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
60 1 0 0 9 1 755 1 0 0 0 0 0 0 0 0 0 0 0 0 0;
61 0 0 0 0 0 1 755 1 0 0 0 0 0 0 0 0 0 0 0 0;
62 0 0 0 0 0 0 1 755 1 0 0 0 0 0 0 0 0 0 0 0;
63 0 0 0 0 0 0 0 1 754 2 0 0 0 0 0 0 0 0 0 0;
64 0 0 0 0 0 0 0 0 1 753 1 0 0 0 0 0 0 0 0 0;
65 0 0 0 0 0 0 0 0 0 1 756 0 0 0 0 0 0 0 0 0;
66 0 0 0 0 0 0 0 0 0 0 1 756 0 0 0 0 0 0 0 0;
67 0 0 0 0 0 0 0 0 0 0 0 1 755 1 0 0 0 0 0 0;
68 0 0 0 0 0 0 0 0 0 0 0 0 1 754 1 0 0 0 0 0;
69 0 0 0 0 0 0 0 0 0 0 0 0 0 1 755 1 0 0 0 0;
70 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 754 1 0 0 0;
71 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 755 1 0 0;
72 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 755 1;
73 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 755];
74
75 [precision, recall, f1_scores, weighted_f1] = compute_f1_scores(confMatrix);
76
Command Window
0.9997
Weighted F1-score: 0.99569
fx >>
    
```

F1 Score(Fine KNN)

```

1 % Define confusion matrix (Replace with actual values)
2 conf_matrix = [
3 753, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
4 0, 755, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
5 1, 0, 753, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
6 2, 1, 0, 740, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
7 0, 0, 0, 1, 755, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
8 1, 0, 0, 0, 0, 755, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
9 2, 0, 0, 0, 0, 0, 0, 754, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
10 ];
11
12 num_classes = size(conf_matrix, 1);
13 precision = zeros(1, num_classes);
14 recall = zeros(1, num_classes);
15 f1_scores = zeros(1, num_classes);
16
17 % Support (Total instances per class)
18 support = sum(conf_matrix, 2);
19 total_samples = sum(support);
20
21 for i = 1:num_classes
22 TP = conf_matrix(i, i); % True Positives
23 FP = sum(conf_matrix(:, i)) - TP; % False Positives
24 FN = sum(conf_matrix(i, :)) - TP; % False Negatives
25
26 precision(i) = TP / (TP + FP + 1e-9); % Avoid division by zero
27 recall(i) = TP / (TP + FN + 1e-9);
28 f1_scores(i) = 2 * (precision(i) * recall(i)) / (precision(i) + recall(i) + 1e-9);
29
Command Window
Weighted F1-score: 0.9990
Weighted F1-score: 0.9993
Weighted F1-score: 0.9997
fx >>
    
```

F1 Score(Linear SVM)



ROC AUC Score(Multiclass Data)

Conclusion

From the above results we can conclude that Fine Tree and Fine KNN give the best possible classification from the other two datasets for multiclass dataset

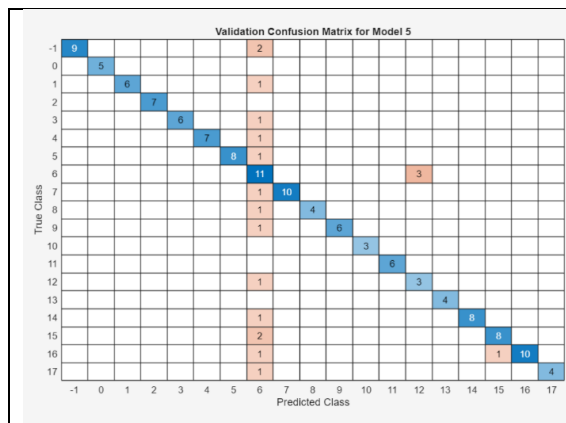
Machine Learning -Fault classification in Robot Joints

4.) Multiclass Subset Data

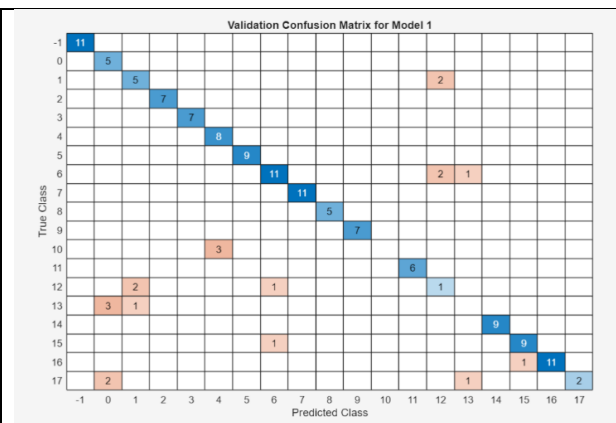
d) Multiclass subset classification in fault detection aims to differentiate between different types of faults in hexapod robot joints and below table gives the results: -

Classification Types	Accuracy	F1-Score	AUC-ROC
Fine tree	86.11	1.000	1.000
Linear SVM	90.97	0.9987	0.985
Fine KNN	73.61	0.9882	0.9660
Boosted Trees	86.51	0.9000	0.9436

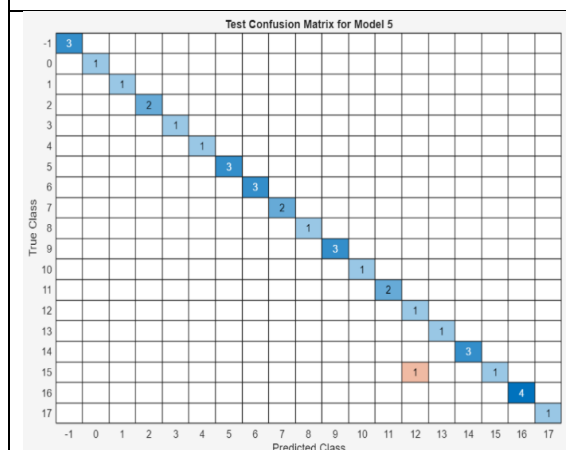
d) Similarly, we have mentioned the confusion matrix and f1 scores for the two best classification methods used for multiclass subset dataset



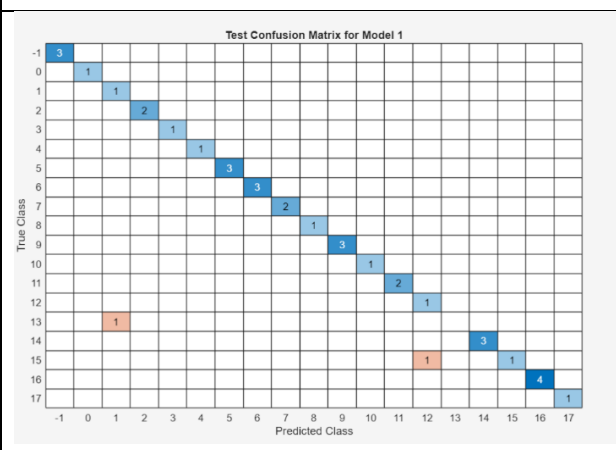
Validation Confusion Matrix(Boosted Trees)



Validation Confusion Matrix(Fine Trees)



Test Confusion Matrix(Boosted Trees)



Test Confusion Matrix(Fine Trees)

Machine Learning -Fault classification in Robot Joints

```

1 % Confusion matrix extracted from the image
2 conf_matrix = [753, 0, 0, 2, 1, 0, 0, 0, 0, 0;
3               0, 744, 3, 1, 1, 0, 0, 0, 0, 0;
4               0, 0, 744, 0, 6, 0, 0, 0, 0, 0;
5               0, 0, 0, 750, 6, 0, 0, 0, 0, 0;
6               0, 0, 0, 0, 756, 0, 0, 0, 0, 0;
7               0, 0, 0, 0, 0, 739, 2, 0, 0, 0;
8               0, 0, 0, 0, 0, 2, 744, 1, 0, 0;
9               0, 0, 0, 0, 0, 1, 755, 0, 0, 0;
10              0, 0, 0, 0, 0, 0, 0, 756, 0, 0;
11              0, 0, 0, 0, 0, 0, 0, 0, 756, 0;
12
13 num_classes = size(conf_matrix, 1);
14 precision = zeros(1, num_classes);
15 recall = zeros(1, num_classes);
16 f1_scores = zeros(1, num_classes);
17
18 % Support (Total instances per class)
19 support = sum(conf_matrix, 2);
20 total_samples = sum(support);
21
22 for i = 1:num_classes
23     TP = sum(conf_matrix(i, :)); % True Positives
24     FP = sum(conf_matrix(:, i)) - TP; % False Positives
25     FN = sum(conf_matrix(i, :)) - TP; % False Negatives
26
27     precision(i) = TP / (TP + FP + 1e-9); % Avoid division by zero
28     recall(i) = TP / (TP + FN + 1e-9);

```

Command Window

```

Weighted F1-score: 0.9987
Weighted F1-score: 1.0000
Weighted F1-score: 0.9682

```

F1 Score(Boosted Trees)

```

1 % Confusion matrix extracted from the image
2 conf_matrix = [11, 0, 0, 5, 0, 0, 0, 0, 0, 0;
3               0, 5, 0, 7, 0, 0, 0, 0, 0, 0;
4               0, 0, 7, 0, 8, 0, 0, 0, 0, 0;
5               0, 0, 0, 8, 9, 0, 0, 0, 0, 0;
6               0, 0, 0, 0, 11, 0, 0, 0, 0, 0;
7               0, 0, 0, 0, 0, 11, 0, 0, 0, 2;
8               0, 0, 0, 0, 0, 0, 5, 0, 0, 0;
9               0, 0, 0, 0, 0, 0, 0, 7, 0, 0;
10              0, 0, 0, 0, 0, 0, 0, 0, 6, 0;
11              0, 0, 0, 0, 0, 0, 0, 0, 0, 9;
12
13 num_classes = size(conf_matrix, 1);
14 precision = zeros(1, num_classes);
15 recall = zeros(1, num_classes);
16 f1_scores = zeros(1, num_classes);
17
18 % Support (Total instances per class)
19 support = sum(conf_matrix, 2);
20 total_samples = sum(support);
21
22 for i = 1:num_classes
23     TP = conf_matrix(i, i); % True Positives
24     FP = sum(conf_matrix(:, i)) - TP; % False Positives
25     FN = sum(conf_matrix(i, :)) - TP; % False Negatives
26
27     precision(i) = TP / (TP + FP + 1e-9); % Avoid division by zero
28     recall(i) = TP / (TP + FN + 1e-9);

```

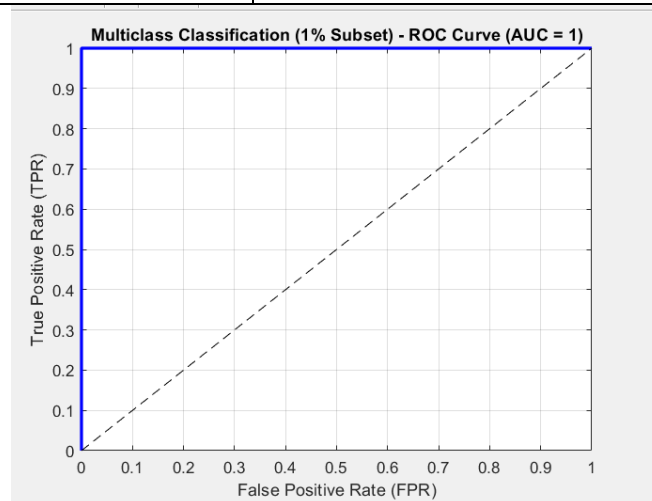
Command Window

```

Weighted F1-score: 1.0000
Weighted F1-score: 1.0000
Weighted F1-score: 0.9000

```

F1 Score(Fine Trees)



ROC AUC Score(Multiclass Subset Data)

Conclusion

From the above results we can conclude that Linear SVM and Boosted Trees (Ensemble) give the best possible classification apart from the other two methods for multiclass subset data

3)Evaluation (K-Fold Cross Validation)

For K-Fold Cross evaluation we put the value of K=5 and K=10 and compared the results and parameter configurations and these are the results we obtained

1.) Binary Data

Model	Avg. Accuracy	Std Dev	Avg Cost
Fine Trees	99.98	0.01	1.00
Linear SVM	94.74	0	0
Fine KNN	99.97	0.01	2.00
Boosted Trees	94.74	0	0

Machine Learning -Fault classification in Robot Joints

Session: untitled
 Training Data: binaryData Observations: 14364 Predictors: 9 Response Name: Label Response Classes: 2
 Validation: 5-fold cross-validation
 Test Data: binaryData Observations: 3591

Select Columns Plot Results Export to Workspace Export to File

Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	99.98 %	3	100.00 %	0
☐	2	SVM	Tested	94.74 %	756	94.74 %	189
☐	3	KNN	Tested	99.99 %	6	100.00 %	0
☐	4	Ensemble	Tested	94.74 %	756	94.74 %	189

Result Table for Binary Dataset (K = 5)

Model 1 Model 2 Model 3 Model 4 Compare Results 1 x Results Table x

Session: ClassificationLearnerSession 2
 Training Data: binaryData Observations: 14364 Predictors: 9 Response Name: Label Response Classes: 2
 Validation: 10-fold cross-validation
 Test Data: binaryData Observations: 3591

Select Columns Plot Results Export to Workspace Export

Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	99.99 %	2	100.00 %	0
☐	2	SVM	Tested	94.74 %	756	94.74 %	189
☐	3	KNN	Tested	99.97 %	4	100.00 %	0
☐	4	Ensemble	Tested	94.74 %	756	94.74 %	189

Result Table for Binary Dataset (K = 10)

Conclusion:-

From the above results we conclude that Fine KNN and Fine Trees perform better for K Folds = 5 and K Folds = 10 for the binary dataset

2.) Binary Subset

Model	Avg. Accuracy	Std Dev	Avg Cost
Fine Tree	99.45	00.69	0
Linear Discriminant	92.36	01.39	14.5
Linear SVM	95.45	04.86	9.00
Boosted Trees	93.02	00.69	12.5

Machine Learning -Fault classification in Robot Joints

Model 1 Model 2 Model 3 Model 4 Compare Results 1 x Results Table x

Session: untitled
 Training Data: binaryDataSubset2 Observations: 144 Predictors: 9 Response Name: Label Response Classes: 2
 Validation: 5-fold cross-validation
 Test Data: binaryDataSubset2 Observations: 36

Select Columns Plot Results Export to Workspace Export to

Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	99.31 %	1	100.00 %	0
☐	2	Discriminant	Tested	93.06 %	10	91.67 %	3
☐	3	KNN	Tested	97.92 %	3	94.44 %	2
☐	4	Ensemble	Tested	93.75 %	9	91.67 %	3

Data set: binaryDataSubset2 Observations: 144 Size: 17 kB Predictors: 9 Response: Label Response Classes: 2 Validation: 5-fold cross-validation Test Data set: binaryDataSubset2

Result Table for Binary Subset (K = 5)

Model 1 Model 2 Model 3 Model 4 Compare Results 1 x Results Table x

Session: untitled
 Training Data: binaryDataSubset2 Observations: 144 Predictors: 9 Response Name: Label Response Classes: 2
 Validation: 10-fold cross-validation
 Test Data: binaryDataSubset2 Observations: 36

Select Columns Plot Results Export to Workspace Export to

Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	100.00 %	0	100.00 %	0
☐	2	Discriminant	Tested	91.67 %	12	88.89 %	4
☐	3	SVM	Tested	93.06 %	10	91.67 %	3
☐	4	Ensemble	Tested	93.06 %	10	91.67 %	3

Result Table for Binary Subset (K = 10)

From the above results we conclude that Linear SVM and Fine Trees perform better for K Folds = 5 and K Folds = 10 for the binary subset

3.) Multiclass Dataset

Model	Avg. Accuracy	Std Dev	Avg Cost
Fine Tree	99.79	00.20	7.5
Linear SVM	97.52	00.61	85
Fine KNN	99.63	00.17	13
Boosted Trees	98.49	00.67	53

Machine Learning -Fault classification in Robot Joints

Session: untitled								
Training Data: multiclassData Observations: 14364 Predictors: 9 Response Name: Label Response Classes: 19								
Validation: 5-fold cross-validation								
Test Data: multiclassData Observations: 3591								
Select Columns Plot Results Export to Workspace Export to File								
Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)	
☐	2	Tree	Tested	99.85 %	22	99.69 %		11
☐	3	SVM	Tested	97.98 %	290	97.94 %		74
☐	4	KNN	Tested	99.61 %	56	99.72 %		10
☐	5	Ensemble	Tested	98.58 %	204	98.19 %		65

Data set: multiclassData Observations: 14364 Size: 1 MB Predictors: 9 Response: Label Response Classes: 19 Validation: 5-fold cross-validation Test Data set: multiclassData

Result Table for Multiclass Dataset(K = 5)

Classification Learner - ClassificationLearnerSession4								
LEARN TEST EXPLAIN								
Test Data Selected Test All Confusion Matrix (Test) ROC Curve (Test) Precision-Recall Curve (Test) Compare Results Results Table Export Plot + EXPORT								
Models								
Sort by Model Number								
1 Tree Accuracy (Test): 99.9% 9/9 features Last change: Fine Tree								
2 SVM Accuracy (Test): 97.3% 9/9 features Last change: Linear SVM								
3 KNN Accuracy (Test): 99.6% 9/9 features Last change: Fine KNN								
4 Ensemble Accuracy (Test): 98.9% 9/9 features Last change: Boosted Trees								
Session: ClassificationLearnerSession4								
Training Data: multiclassData Observations: 14364 Predictors: 9 Response Name: Label Response Classes: 19								
Validation: 10-fold cross-validation								
Test Data: multiclassData Observations: 3591								
Select Columns Plot Results Export to Workspace Export to File								
Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)	
☐	1	Tree	Tested	99.76 %	34	99.89 %		4
☐	2	SVM	Tested	98.07 %	277	97.33 %		96
☐	3	KNN	Tested	99.60 %	57	99.55 %		16
☐	4	Ensemble	Tested	98.14 %	267	98.86 %		41

Data set: multiclassData Observations: 14364 Size: 1 MB Predictors: 9 Response: Label Response Classes: 19 Validation: 10-fold cross-validation Test Data set: multiclassData

Result Table for Multiclass Dataset(K=10)

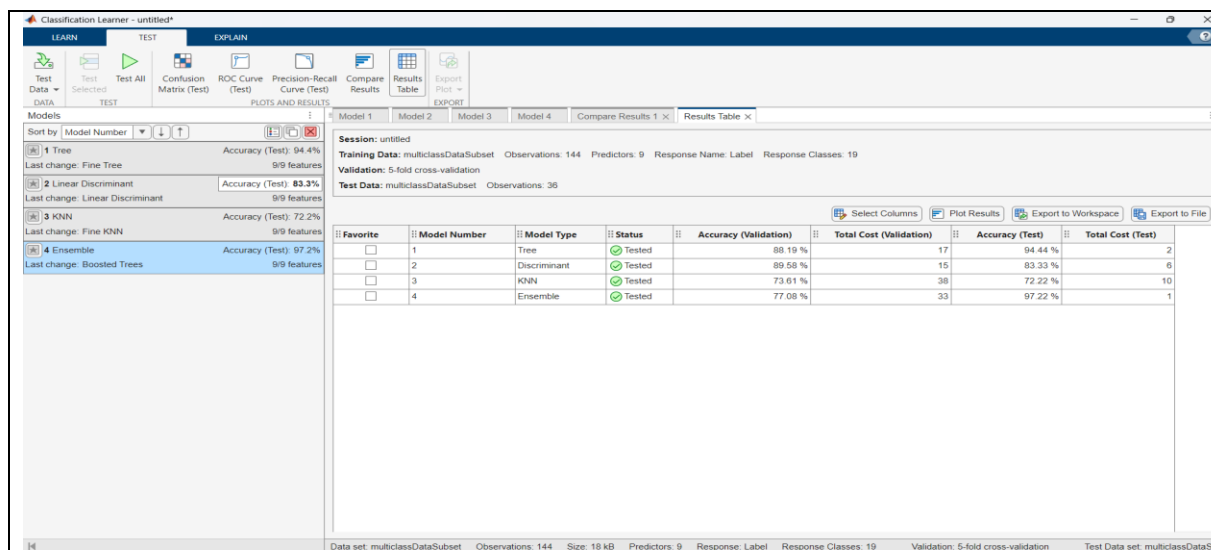
Machine Learning -Fault classification in Robot Joints

Conclusion: -

From the above results we conclude that Fine Trees and Fine KNN perform better for K Folds = 5 and K Folds = 10 for the multiclass dataset

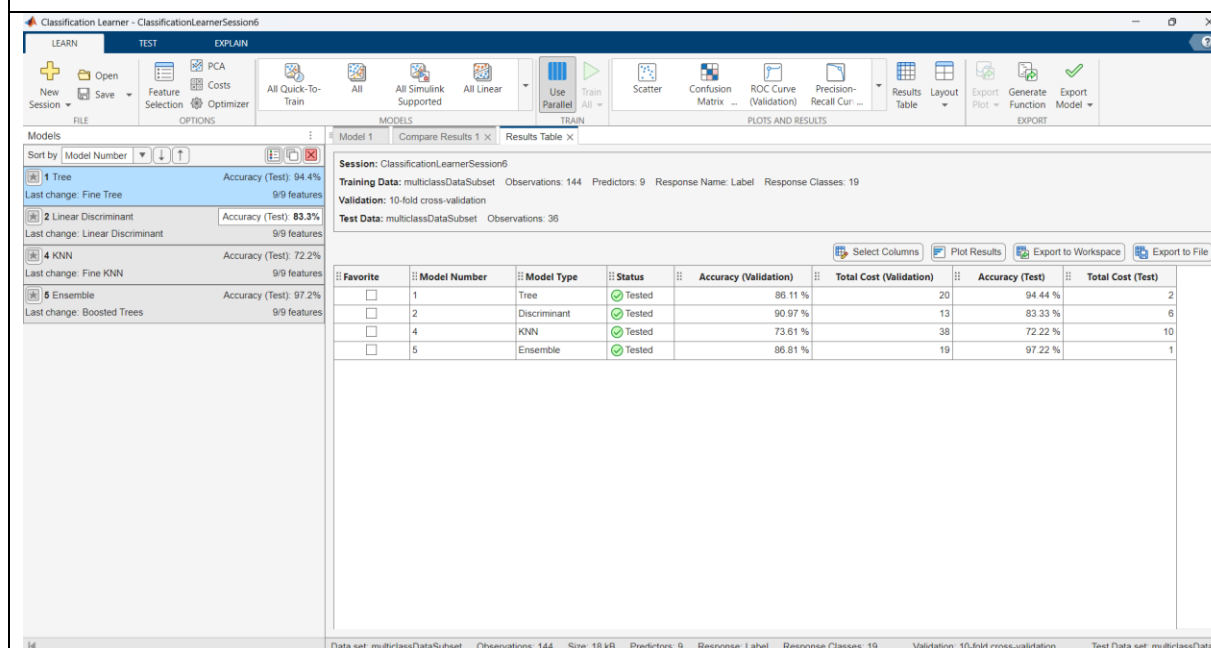
4.) Multiclass Subset

Model	Avg. Accuracy	Std Dev	Avg Cost
Fine Tree	99.79	00.20	7.5
Linear SVM	97.52	00.61	85
Fine KNN	99.63	00.17	13
Boosted Trees	98.49	00.67	53



Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	98.19 %	17	94.44 %	2
☐	2	Discriminant	Tested	89.58 %	15	83.33 %	6
☐	3	KNN	Tested	73.61 %	38	72.22 %	10
☐	4	Ensemble	Tested	77.08 %	33	97.22 %	1

Result table for multiclass subset(K=5)



Favorite	Model Number	Model Type	Status	Accuracy (Validation)	Total Cost (Validation)	Accuracy (Test)	Total Cost (Test)
☐	1	Tree	Tested	86.11 %	20	94.44 %	2
☐	2	Discriminant	Tested	90.97 %	13	83.33 %	6
☐	4	KNN	Tested	73.61 %	38	72.22 %	10
☐	5	Ensemble	Tested	86.81 %	19	97.22 %	1

Result Table for multiclass subset(K=10)

Conclusion

From the above results we conclude that Linear Discriminant (LDA) and Boosted Trees (Ensemble) perform better for K Folds = 5 and K Folds = 10 for the multiclass subset

PART IV – Classification using Neural Networks

While working with datasets containing multiple faults in the joints of a hexapod robot, we discovered that patternNet is an ideal choice for our coursework on fault classification. It effectively integrates with feature extraction techniques, offering multiple hidden layers that support hierarchical feature representation. Additionally, it's flexible network topology allows for easy adjustments to the number of neurons and layers. The patternnet function is specifically designed for classification tasks, with the objective of assigning input patterns to predefined categories.

The neural network used for pattern recognition is trained using the patternnet function.

“patternnet(hiddenLayerSize)”

patternnet is specifically designed for **pattern recognition and classification problems**. This function provides an efficient framework for training **feedforward neural networks using backpropagation**.

- Binary Classification (Fault/No Fault):

In our dataset, we need to determine whether a joint has a fault or not. The patternnet function effectively learns patterns in the data and assigns them to one of the two categories.

- Multi-Class Classification (Different Fault Types):

As our dataset contains multiple fault categories, patternnet can be adapted to handle multi-class classification by using one-hot encoding for the target labels.

Normalising data

Neural networks perform better when input data is normalized.

Adding $X = \text{normalize}(X)$; to the MATLAB code improves the performance of the neural network by ensuring that all features are on a similar scale.

Without normalisation, the data may have features with vastly different magnitudes i.e. features with large values dominate learning. Also, the neural network may struggle to adjust weights efficiently and it might take longer time to reach an optimal state.

After Adding Normalization, each feature is scaled so that it has:

- A mean of 0
- A standard deviation of 1

Now, all input features are on the same scale, preventing larger features from dominating. This allows the network to learn efficiently and provides optimal output.

Machine Learning -Fault classification in Robot Joints

Binary Data

Below, we experimented with binary data with patternnet and trainNetwork, and after considering imbalanced data, we trained the patternnet network on an equal number of fault and no-fault samples of binary data to get better results.

By experimenting with different training splits and hyperparameters and evaluating the classification approaches using metrics such as ROC AUC and a confusion matrix, we get the following results,

- Training with patternnet

Hyperparameters used are as follows,

- Epochs
It helps to determine the no. of times the model learns from the training data.
- Learning rate
It controls the step size at which the model updates its weights during training
- Neural Layers
The no. of hidden layers and neurons for each layer determines the model's ability to learn complex patterns

We determine the AUC (Area under the curve) Value represents ability of the model to distinguish between classes.

AUC = 1.0 → Perfect model (ideal classifier).

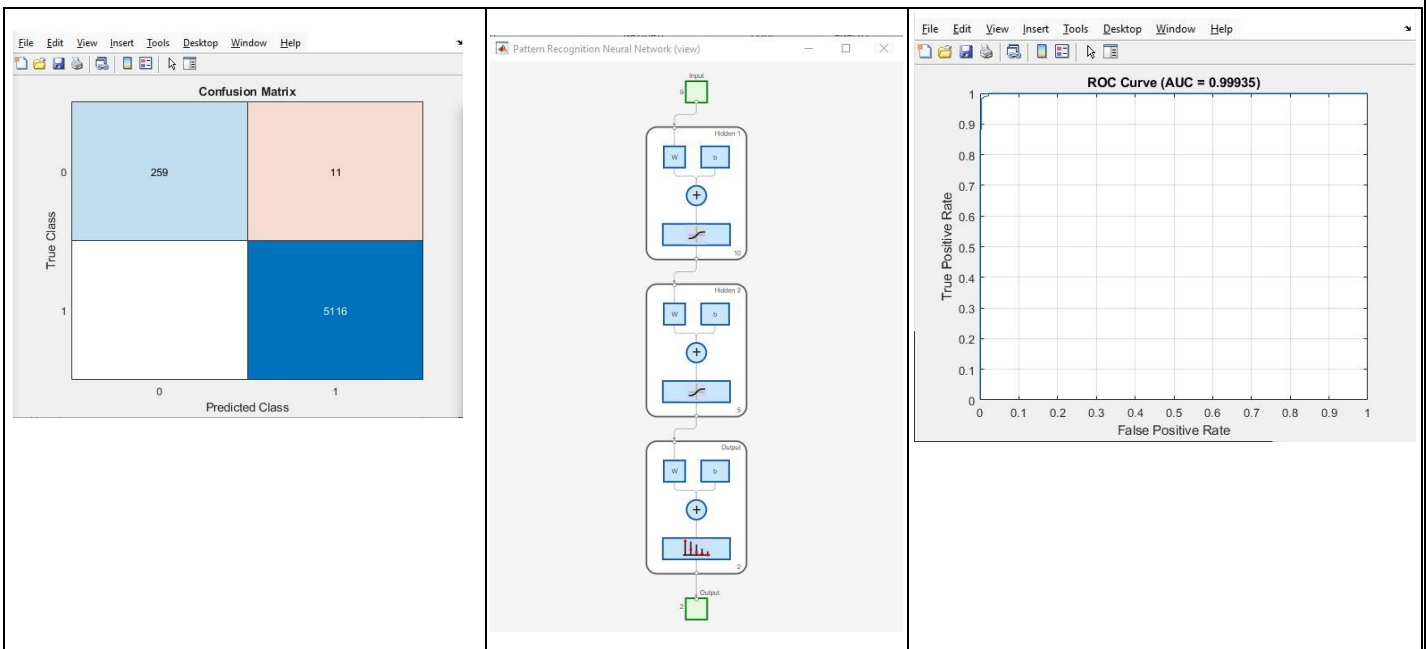
AUC = 0.5 → Random guessing (no predictive power).

AUC < 0.5 → Worse than random (model is making incorrect predictions)

We trained with different training splits 70/30 and 80/20 using the patternnet function.

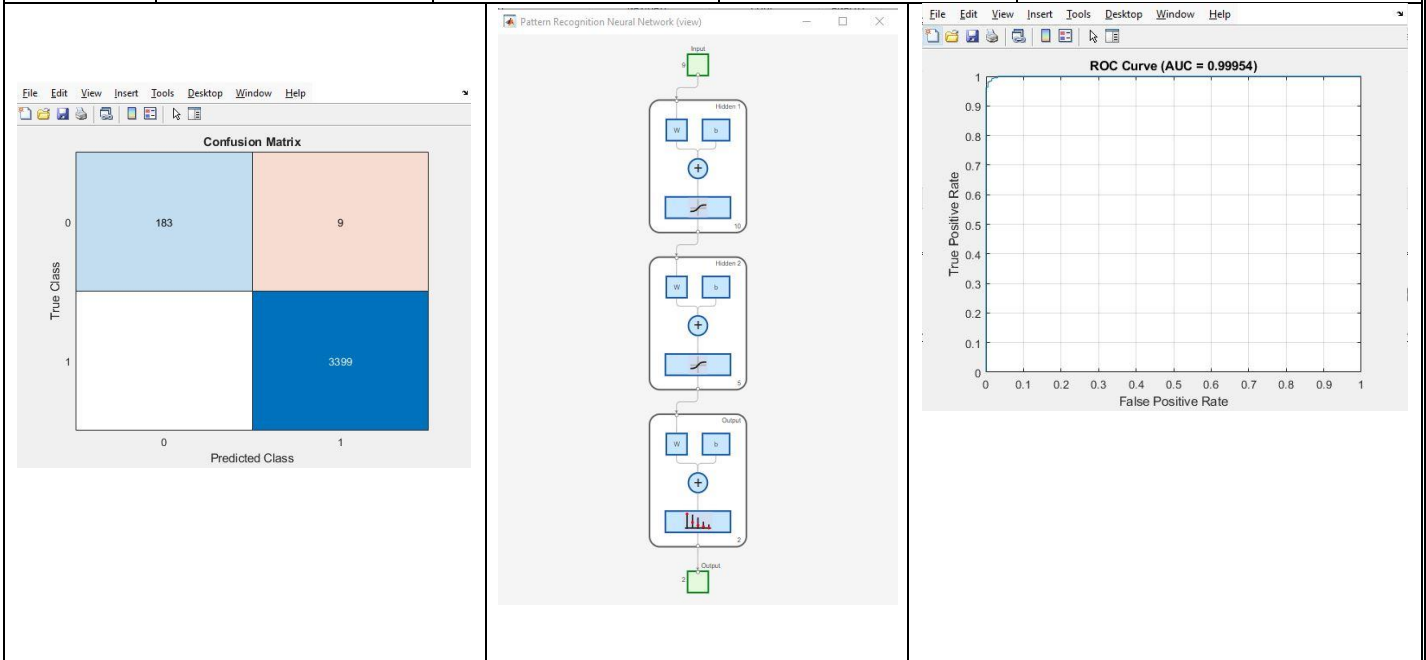
Training Splits in Percentage 70/30%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.866
		100	0.01	0.9864
		200	0.001	0.853
		300	0.0001	0.992
	2	50	0.1	0.9241
		100	0.01	0.9917
		200	0.001	0.9987
		300	0.0001	0.99935

Machine Learning -Fault classification in Robot Joints



Training Splits in Percentage 80/20%

Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.9418
		100	0.01	0.9758
		200	0.001	0.8694
		300	0.0001	0.99682
	2	50	0.1	0.9624
		100	0.01	0.9982
		200	0.001	0.99954
		300	0.0001	0.9872



Machine Learning -Fault classification in Robot Joints

Analysis of Underfitting and Overfitting in our Results

Potential underfitting in your results:

- **At 50 epochs, 1-layer models:**
 - 70/30 split: AUC = **0.866** (low)
 - 80/20 split: AUC = **0.9418** (better but still lower than deeper models)
- **At 0.1 learning rate across all epochs and splits:**
 - AUC values are consistently lower compared to smaller learning rates.
 - Example:
 - **70/30 split, 1-layer, 50 epochs, 0.1 LR → AUC = 0.866**
 - **80/20 split, 1-layer, 50 epochs, 0.1 LR → AUC = 0.9418**
 - The high learning rate may have prevented convergence.

Conclusion on underfitting:

- **1-layer models are underfitting** at lower epochs.
- **High learning rates (0.1) lead to underfitting** as they prevent proper weight updates.

2. Signs of Overfitting

Potential overfitting in our results:

- **For 2-layer models with 300 epochs, 0.0001 LR:**
 - 70/30 split: **AUC = 0.99935** (very close to 1.0)
 - 80/20 split: **AUC = 0.9872** (slightly lower)
 - The high AUC suggests the model may have memorized training data.
- **Higher AUC with a larger training split (80/20):**
 - 80/20 split should generalize better due to more training data, yet the **AUC drops slightly (0.99935 → 0.9872)** for deep models.
 - This drop suggests that **the 70/30 model is overfitting** because it performs worse when tested on slightly more unseen data.

Conclusion on overfitting:

- **2-layer models with 300 epochs and 0.0001 LR might be overfitting** (AUC ~0.999).
- **Slight AUC drop from 70/30 to 80/20 split suggests possible overfitting.**

Underfitting: Found in shallow (1-layer) models at fewer epochs (50) and high learning rates (0.1).

Machine Learning -Fault classification in Robot Joints

Overfitting: Likely in deep (2-layer) models with many epochs (300) and small learning rates (0.0001), as AUC is too high (~0.999).

- **Training with trainNetwork**

trainNetwork is a MATLAB function used to train deep learning models, including neural networks, with various types of layer architectures. It is a flexible function that works with different types of models, including feedforward neural networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and more.

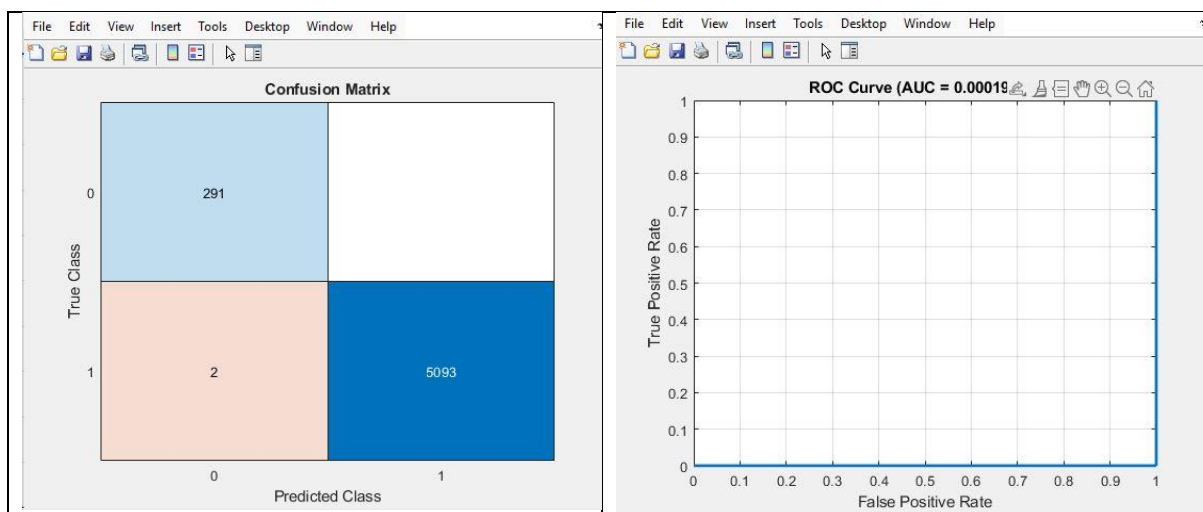
- trainNetwork allows us to define custom structures. We can specify various layers, activation functions, batch normalization, and dropout layers.
- The network cooperates with Adam, RMSprop, SGDM (Stochastic Gradient Descent with Momentum), and others to aid in convergence.
- It can train models in mini batches, reducing memory usage for large datasets.

Binary data has been trained with a custom network to check for better result, but the AUC value gives poor result when trained with different parameters.

Layers used in the network

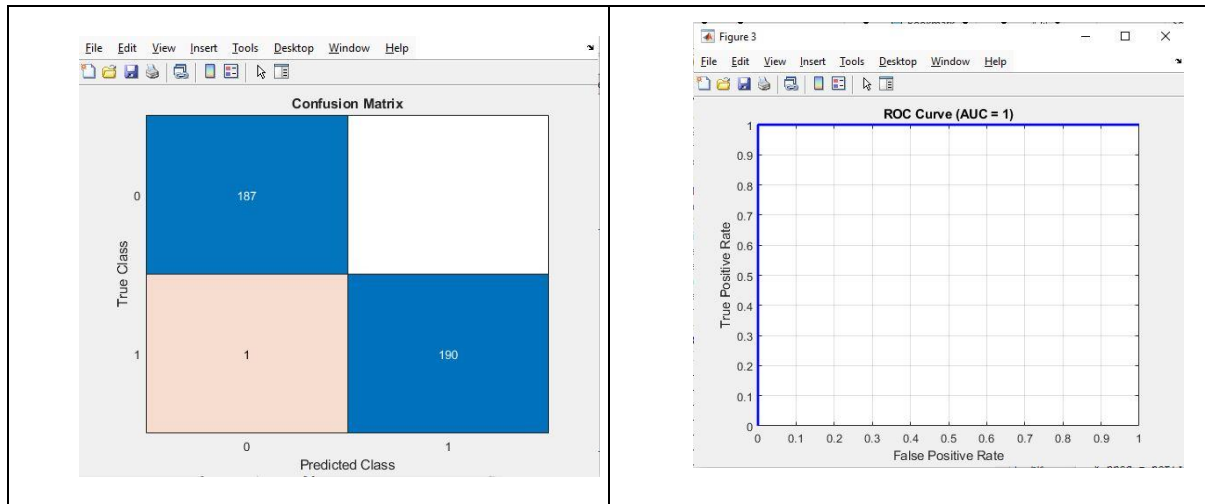
```
layers = [  
    featureInputLayer(size(X, 2)) % Input Layer  
    fullyConnectedLayer(10)      % First hidden layer (10 neurons)  
    reluLayer                    % ReLU activation  
    fullyConnectedLayer(5)      % Second hidden layer (5 neurons)  
    reluLayer                    % ReLU activation  
    fullyConnectedLayer(2)      % Output layer (2 neurons for binary classification)  
    softmaxLayer                 % Converts scores to probabilities  
    classificationLayer          % Defines loss function (cross-entropy)  
];
```

confusion matrix and AUC value obtained from training above network



Machine Learning -Fault classification in Robot Joints

- Training with patternnet using equal sampling of fault and no-fault binary data



For a 2 layered network with 10 and 5 neurons and for an epoch and learning rate of 300 and 0.0001 respectively gives a reliable confusion matrix and AUC value 1.

1% Binary Data

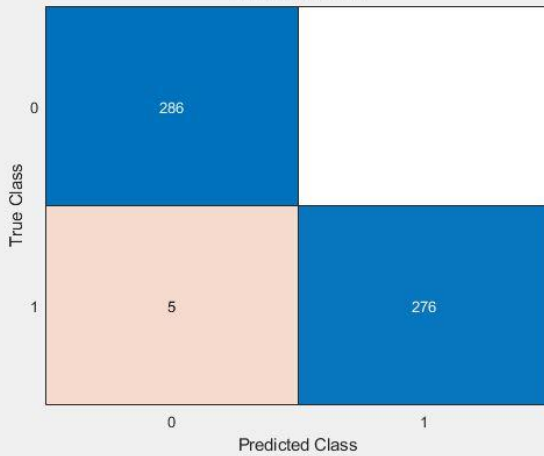
A random sample of 1% of binary data was trained using the patternnet network, we have trained with different training splits using the function of 70/30 and 80/20

Training Splits in Percentage 70/30%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.9047
		100	0.01	0.9821
		200	0.001	0.9788
		300	0.0001	0.9941
	2	50	0.1	0.9258
		100	0.01	0.8086
		200	0.001	0.9997
		300	0.0001	0.9512

Machine Learning -Fault classification in Robot Joints

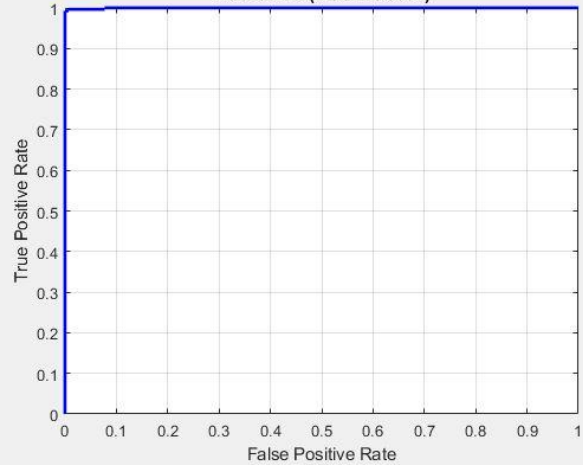
File Edit View Insert Tools Desktop Window Help

Confusion Matrix



File Edit View Insert Tools Desktop Window Help

ROC Curve (AUC = 0.9997)



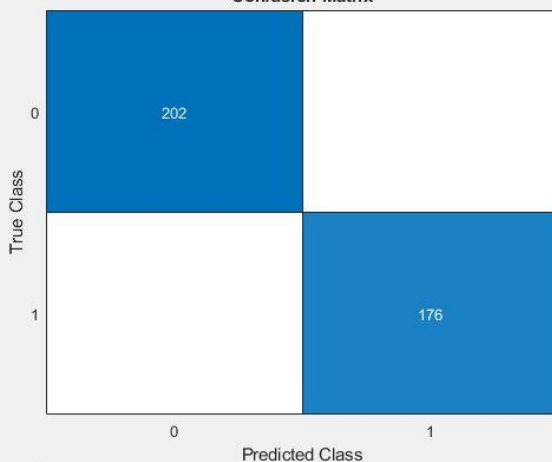
Training Splits in Percentage 80/20%

Hyperparameter

No. of layers	Epochs	Learning rate	AUC Value
1	50	0.1	0.8381
	100	0.01	0.9752
	200	0.001	0.8833
	300	0.0001	0.9950
2	50	0.1	0.8185
	100	0.01	0.9997
	200	0.001	1
	300	0.0001	0.9989

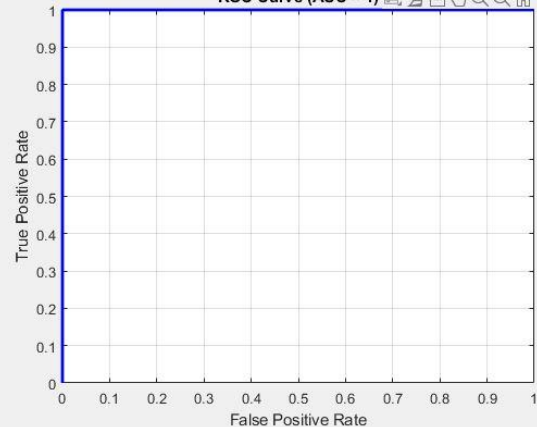
File Edit View Insert Tools Desktop Window Help

Confusion Matrix



File Edit View Insert Tools Desktop Window Help

ROC Curve (AUC = 1)



Machine Learning -Fault classification in Robot Joints

Analysis of Underfitting and Overfitting in Your Results

1. Signs of Underfitting

Potential underfitting in our results:

- 1-layer, 50 epochs, 0.1 learning rate (both splits):
 - 70/30 split: AUC = 0.9047
 - 80/20 split: AUC = 0.8381 (lower than 70/30)
- 2-layer, 50 epochs, 0.1 learning rate:
 - 70/30 split: AUC = 0.9258
 - 80/20 split: AUC = 0.8185 (lower than 1-layer model)
 - Suggests the model is struggling at 50 epochs, possibly not trained enough.

Conclusion on underfitting:

- Underfitting is present at 50 epochs, especially at a 0.1 learning rate.
- Increasing epochs and lowering learning rate improves performance.

2. Signs of Overfitting

Potential overfitting in our results:

- 2-layer, 200 epochs, 0.001 learning rate:
 - 70/30 split: AUC = 0.9997
 - 80/20 split: AUC = 1.0 (perfect score, which is unrealistic)
 - AUC too high suggests overfitting to the training data.
- 2-layer, 300 epochs, 0.0001 learning rate:
 - 70/30 split: AUC = 0.9512
 - 80/20 split: AUC = 0.9989
 - The model is likely overfitting with deep layers and small learning rates.

Conclusion on overfitting:

- Severe overfitting occurs at 2 layers, 200–300 epochs, and 0.001–0.0001 learning rate.
- The 80/20 split results confirm overfitting

Underfitting at 50 epochs and 0.1 learning rate (not enough training).

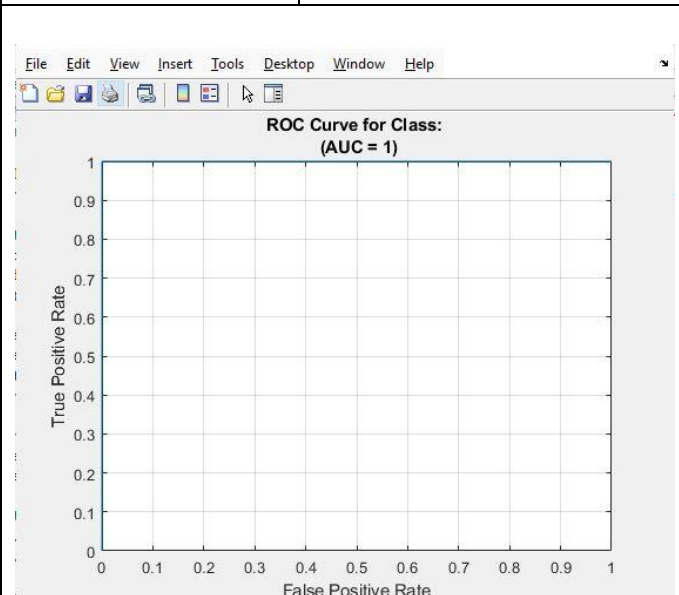
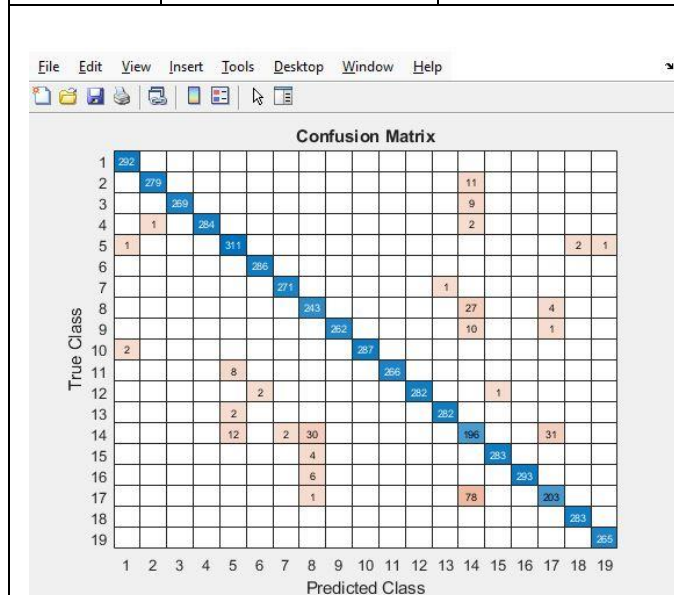
Overfitting at 2 layers, 200+ epochs, and small learning rates (0.001 & 0.0001) (memorizing training data).

Machine Learning -Fault classification in Robot Joints

Multiclass Dataset

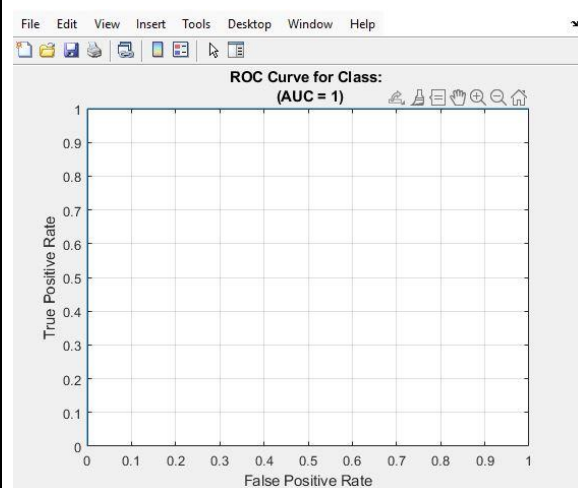
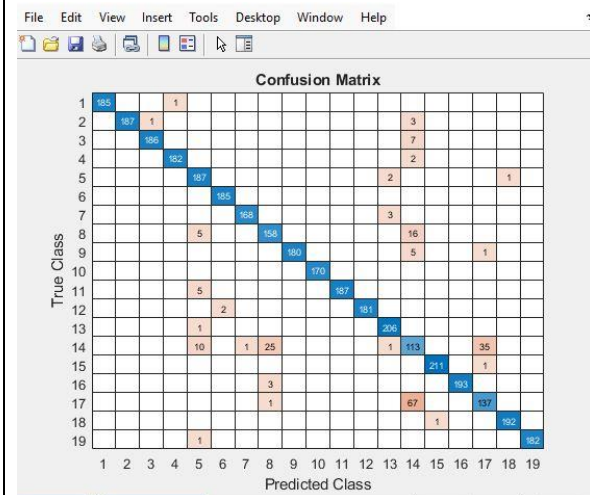
Below we experimented multiclass data with patternnet under different parameter to obtain ROC curve and confusion matrix.

Training Splits in Percentage 70/30%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.9879
		100	0.01	0.99307
		200	0.001	0.99685
		300	0.0001	1
	2	50	0.1	0.9336
		100	0.01	0.9457
		200	0.001	0.9994
		300	0.0001	0.98901



Training Splits in Percentage 80/20%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.9522
		100	0.01	0.9944
		200	0.001	0.9987
		300	0.0001	1
	2	50	0.1	0.8570
		100	0.01	0.93182
		200	0.001	0.98572
		300	0.0001	0.9618

Machine Learning -Fault classification in Robot Joints



Analysis of Underfitting and Overfitting in Your Results

1. Underfitting Analysis

No major underfitting detected:

- The lowest AUC value observed is 0.8570 (80/20 split, 2-layer, 50 epochs, 0.1 learning rate).
- All other values are above 0.9, which indicates the model is learning well.
- Increasing epochs and reducing the learning rate improves AUC values, showing that the model is benefiting from more training.

Conclusion: No clear underfitting issue. The model performs well even at lower epochs and higher learning rates.

2. Overfitting Analysis

Possible overfitting signs:

- 1-layer, 300 epochs, 0.0001 learning rate:
 - 70/30 split: AUC = 1.0
 - 80/20 split: AUC = 1.0
- 2-layer, 200 epochs, 0.001 learning rate:
 - 70/30 split: AUC = 0.9994
 - 80/20 split: AUC = 0.98572
 - Very high AUC, but a slight drop in the 80/20 split, indicating mild overfitting.

Machine Learning -Fault classification in Robot Joints

Conclusion: Overfitting is likely present in deeper models with 200+ epochs and very small learning rates (0.001 or 0.0001). The model may be memorizing training data rather than generalizing.

No underfitting (even the lowest AUC is reasonable).

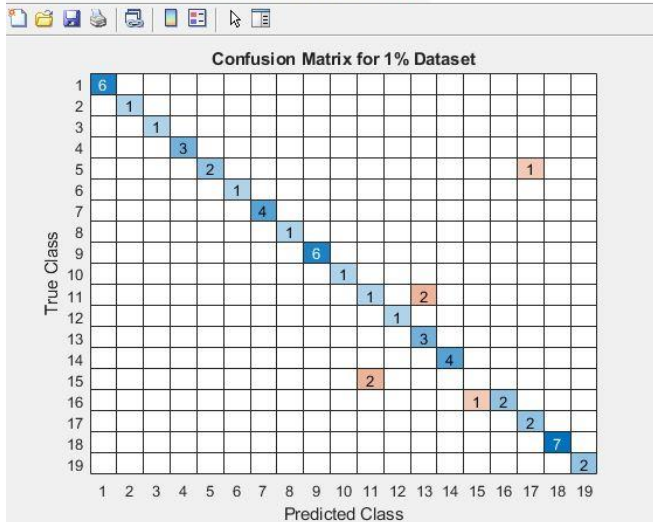
Overfitting is likely in 2-layer, high-epoch, low learning rate settings (e.g., 300 epochs, 0.0001 learning rate).

1% Multiclass Dataset

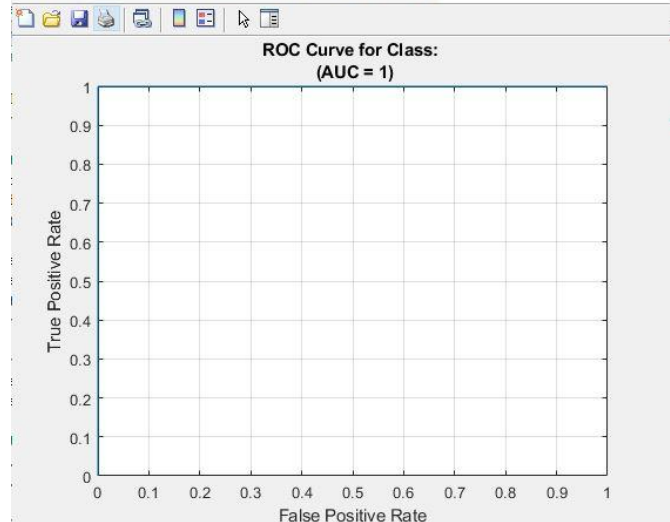
A random sample of 1% of multiclass data was trained using the patternnet network. We trained with different training splits 70/30 and 80/20 using different parameters.

Training Splits in Percentage 70/30%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	0.98693
		100	0.01	1
		200	0.001	0.9869
		300	0.0001	1
	2	50	0.1	1
		100	0.01	1
		200	0.001	1
300		0.0001	0.9934	

FileEditViewInsertToolsDesktopWindowHelp

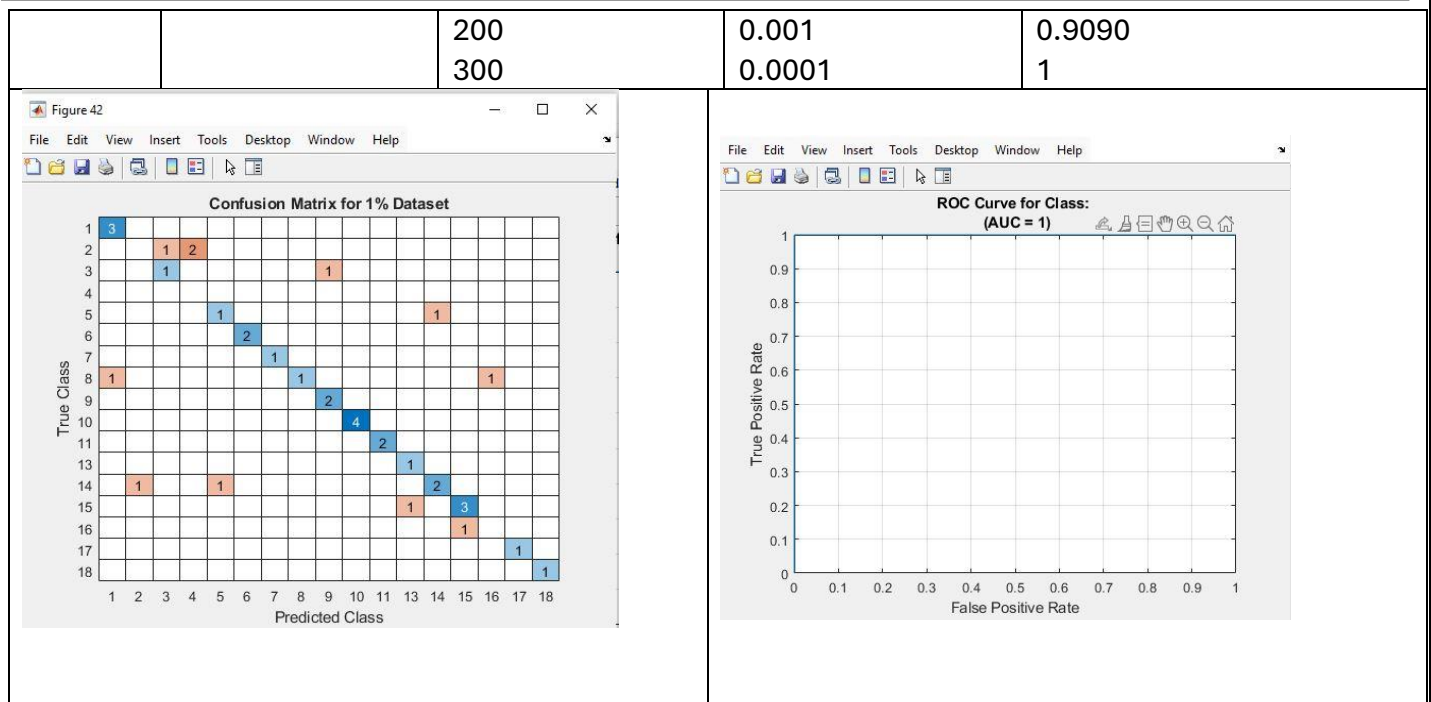


FileEditViewInsertToolsDesktopWindowHelp



Training Splits in Percentage 80/20%				
Hyperparameter	No. of layers	Epochs	Learning rate	AUC Value
	1	50	0.1	1
		100	0.01	0.9411
		200	0.001	1
		300	0.0001	1
	2	50	0.1	1
100		0.01	1	

Machine Learning -Fault classification in Robot Joints



Analysis of Overfitting and Underfitting

Overfitting Analysis

1. AUC = 1 Across Multiple Settings

In both training splits, multiple configurations (especially with 2 layers and higher epochs) achieve AUC = 1.

This suggests that the network memorized the training data, which is a strong indication of overfitting.

2. Effect of Learning Rate and Epochs

In the first split (70/30%), with 2 layers and a learning rate of 0.0001, the model achieves AUC = 1 at 50, 100, and 300 epochs.

Conclusion on Overfitting

- Since multiple configurations reach AUC = 1, it's likely that the network has low bias but high variance, meaning it has memorized training data instead of learning generalizable patterns.

Underfitting Analysis

Indicators of Underfitting in our Results

1. Lower AUC at Specific Epochs

For the 1-layer model at 100 epochs with 0.01 learning rate, AUC = 0.9411.

Machine Learning -Fault classification in Robot Joints

For the 2-layer model at 200 epochs with 0.001 LR, AUC = 0.9090.

These values indicate that the model has not yet fully learned the patterns, meaning it may require more training epochs or a slightly larger network.

2. Performance Variation Based on Learning Rate

At 200 epochs with 0.001 LR, AUC is 0.9090, while at 300 epochs, AUC = 1.

This suggests that the model needed more training to learn the patterns.

3. Comparison of 1-Layer vs. 2-Layer Networks

- The 1-layer network performs slightly worse in some cases (e.g., 0.9411 AUC), meaning it may lack the complexity to fully learn the dataset.
- However, the 2-layer network improves AUC after more epochs, showing that more training was needed to improve performance.

Conclusion on Underfitting

- When AUC is below 1 (e.g., 0.9090, 0.9411), it suggests the model hasn't fully captured the data's complexity.

References

- ❖ Jain, A.K. (2010) 'Data clustering: 50 years beyond K-means', *Pattern Recognition Letters*, 31(8), pp. 651–666. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0167865509002323>. [Accessed 22 March 2025]
- ❖ Ester, M., Kriegel, H.P., Sander, J. and Xu, X. (1996) 'A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise', *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231. Available at: <https://file.biolab.si/papers/1996-DBSCAN-KDD.pdf>. [Accessed 20 March 2025]
- ❖ Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357. Available at: <https://arxiv.org/pdf/1106.1813>. [Accessed 14 March 2025]
- ❖ MathWorks (n.d.) *patternnet*. Available at: <https://www.mathworks.com/help/deeplearning/ref/patternnet.html> [Accessed 10 March 2025].
- ❖ Haykin, S. (2009) *Neural Networks and Learning Machines*. 3rd edn. Upper Saddle River, NJ: Pearson. Available at: <https://dai.fmph.uniba.sk/courses/NN/haykin.neural-networks.3ed.2009.pdf>. [Accessed 15 March 2025]
- ❖ MathWorks (n.d.) *Deep Learning Toolbox User's Guide*. Available at: <https://www.mathworks.com/help/deeplearning/index.html> [Accessed 20 March 2025].

Machine Learning -Fault classification in Robot Joints

- ❖ Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*. New York: Springer. Available at: <https://www.cs.man.ac.uk/~fumie/tmp/bishop.pdf>. [Accessed 10 March 2025]
- ❖ MathWorks (n.d.) *Classification*. Available at: <https://www.mathworks.com/help/stats/classification.html> [Accessed 14 March 2025]
- ❖ Feldesman, M. R. (2002). Classification trees as an alternative to linear discriminant analysis. *American Journal of Physical Anthropology*, 119(3), 257–275

Appendix-MATLAB Codes

The appendix Include all MATLAB code with sufficient comments are as follows.

[APPENDIX.docx](#)

Fault classification in Robot Joints- Matlab Codes

[MATLAB CODES.zip](#)